

Making Predictions

Simple Regression

SIT112 | Intro to Data Science & AI

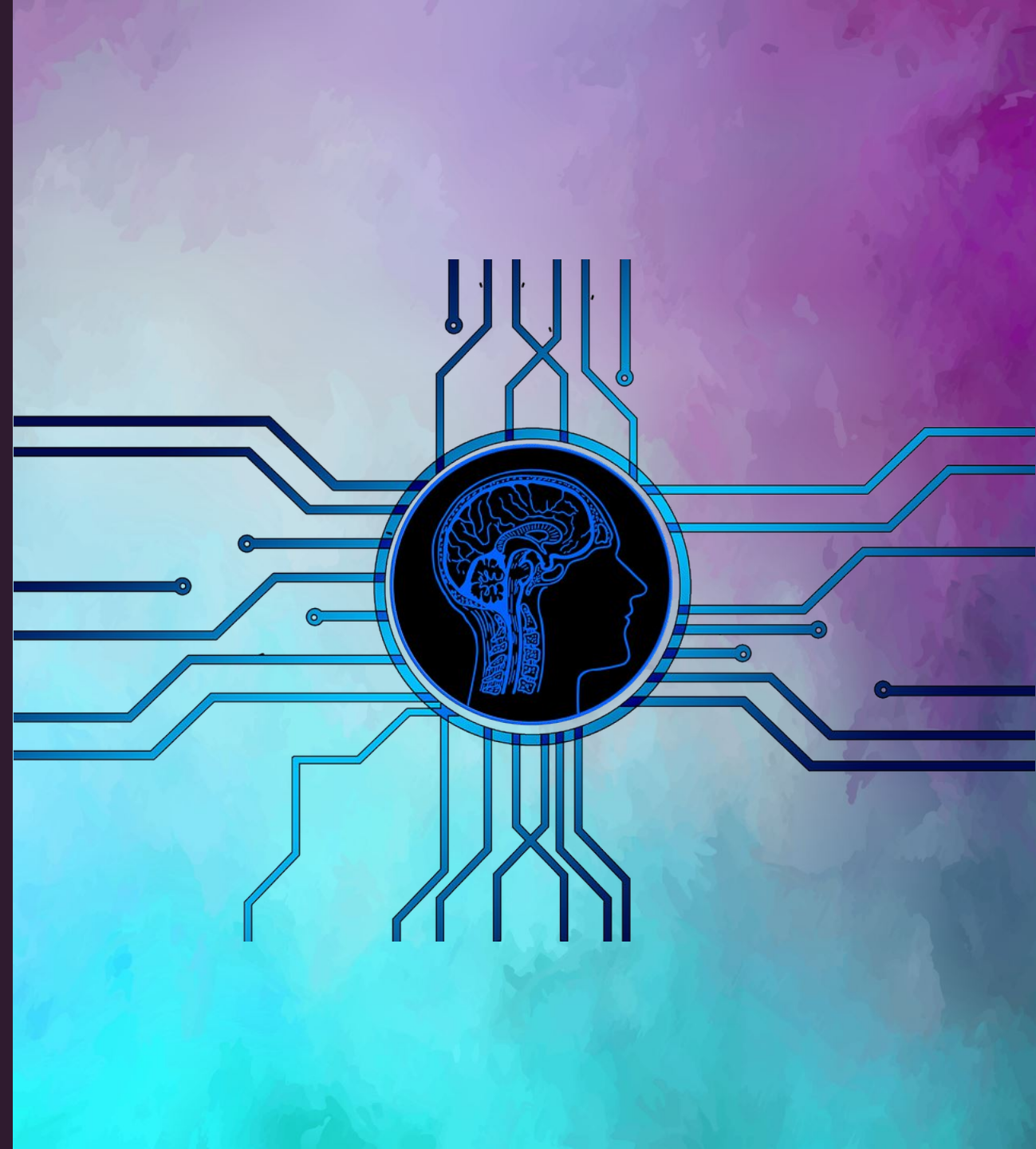
Lecture Week 9

Dr. Fatima Ansarizadeh

Lecturer in Applied AI

School of Information Technology, Deakin University

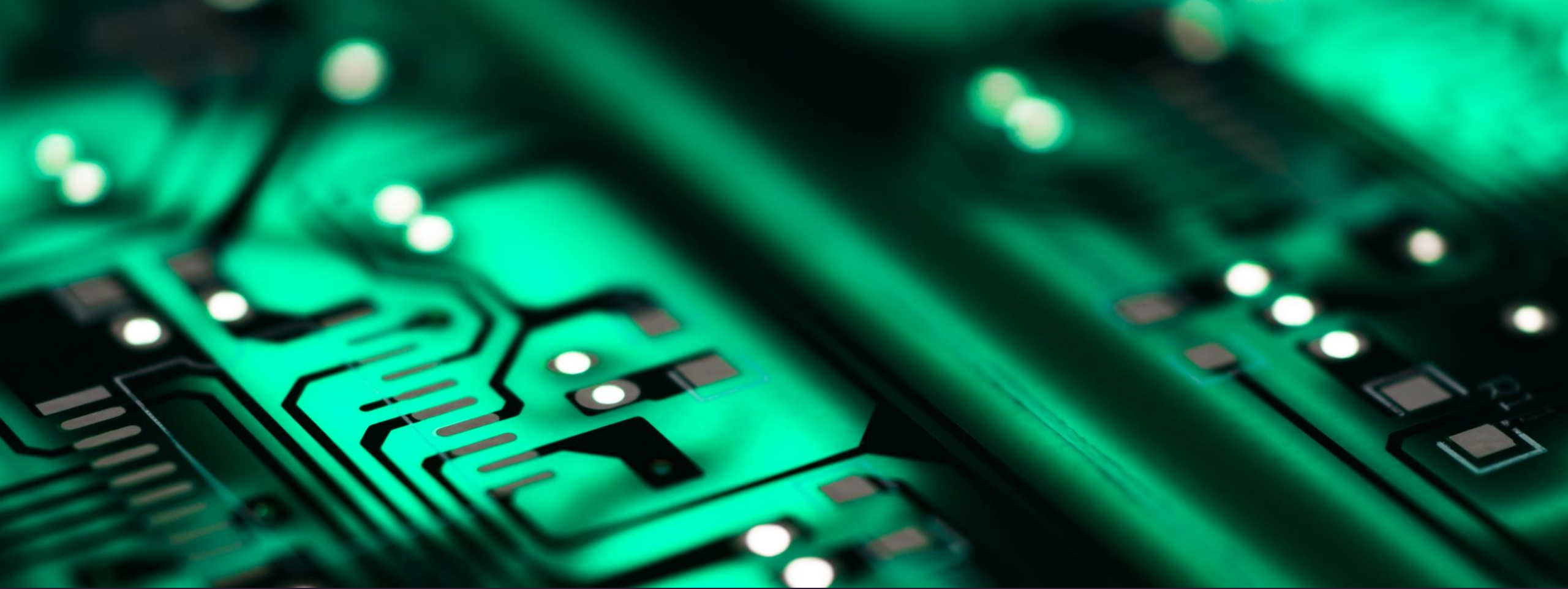
Email: f.ansarizadeh@deakin.edu.au



Objectives



1. Use the `corr()` method to identify correlations.
2. Use scatter plots or heatmaps to identify correlations.
3. Distinguish between a simple regression and a multiple regression.
4. Use Scikit-learn to create and validate a (simple) linear regression model and to use the model to make predictions.
5. Use Seaborn to plot the predicted values and the residuals of a (simple) linear regression model.
6. Use the Seaborn `lplot()` method to create and plot (simple) linear, logistic, polynomial, and lowess regression models.
7. Use the Seaborn `residplot()` method to plot the residuals for (simple) regression models.



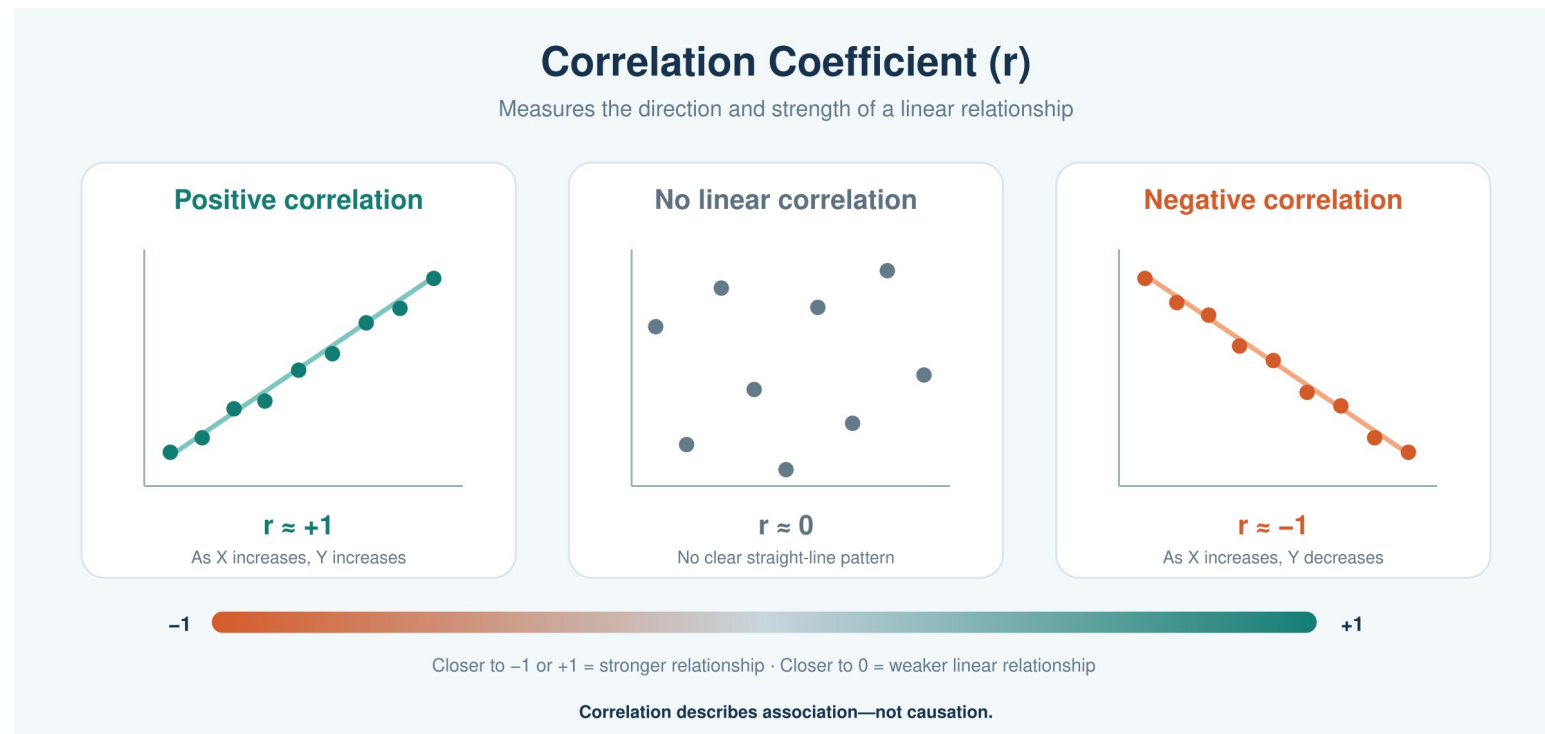
Correlations Between Variables

What is Correlation Coefficient

A **correlation coefficient** is a number that measures the **strength** and **direction** of the relationship between **two numerical** variables.

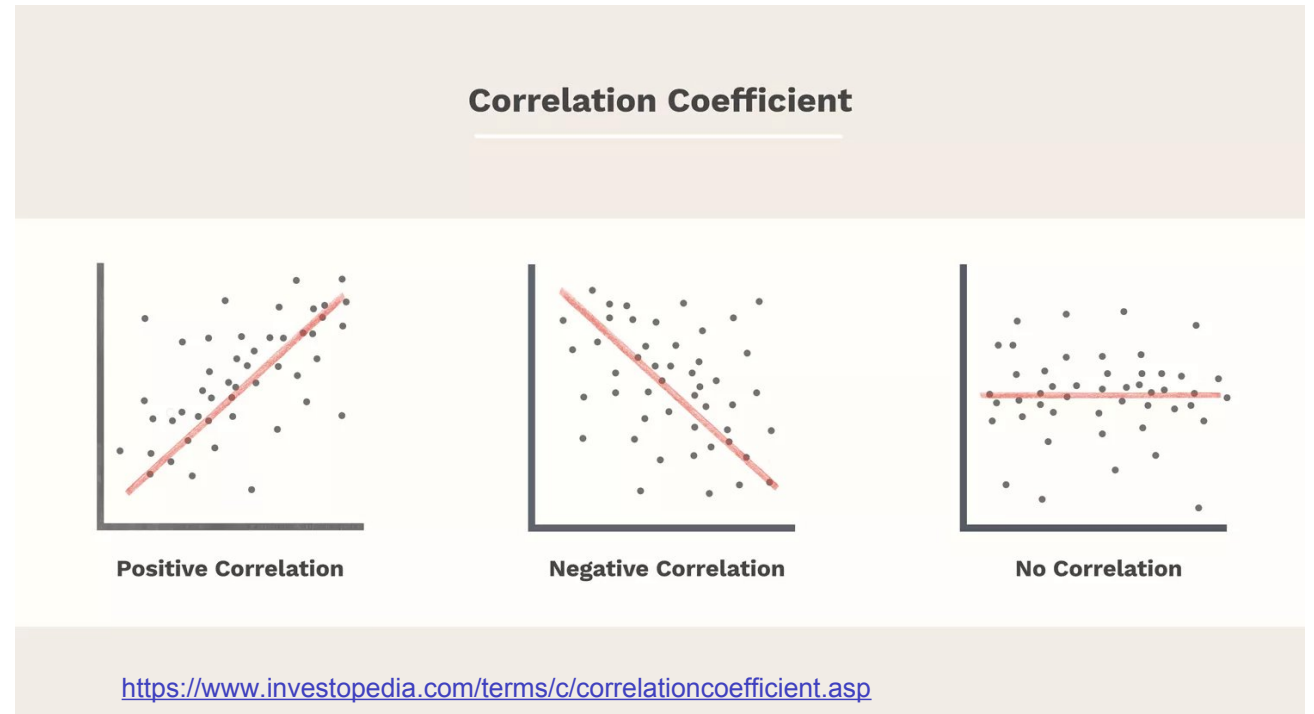
Examples:

- **Positive correlation:** As temperature increases, ice-cream sales tend to increase.
- **Negative correlation:** As product price increases, demand may decrease.
- **No correlation:** Shoe size and exam score are usually unrelated.



What is Correlation Coefficient?

- A statistical measure that indicates the degree to which two variables are related to each other (r-value).
- It represents the strength and direction of a linear relationship between variables.

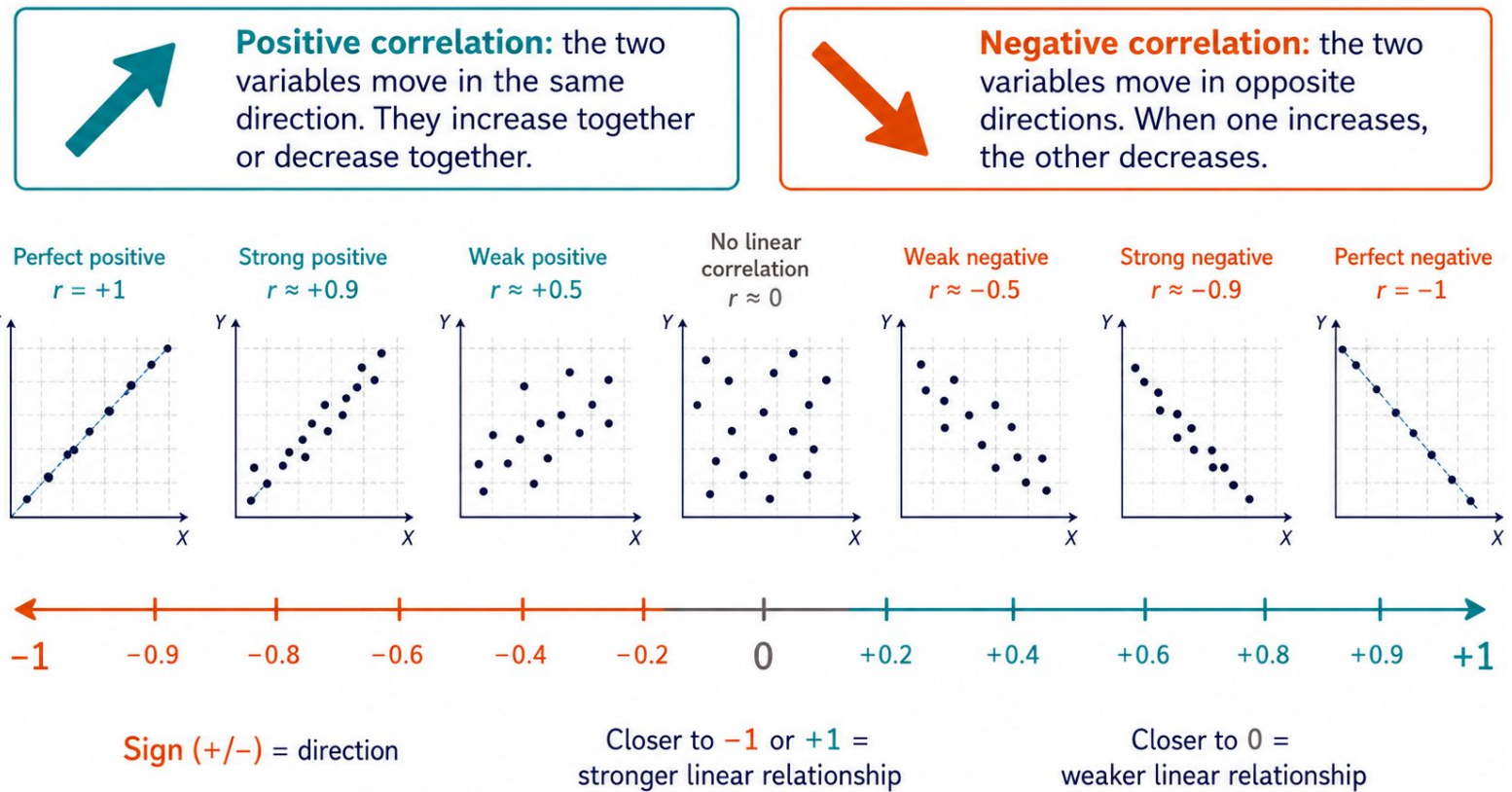


What is Correlation Coefficient? (Cont.)

- E.g., suppose we are studying the relationship between a person's height and weight. We collect data on a sample of individuals and calculate the correlation coefficient between these two variables.
- A positive r-value close to 1, indicates a strong positive relationship, meaning that as height increases, weight tends to increase as well.
- A negative r-value close to -1, indicates a strong negative relationship, meaning that as height increases, weight tends to decrease.
- An r-value close to 0, indicates no linear relationship between height and weight.

The **strength of correlation** describes how closely the data points follow a straight-line pattern.

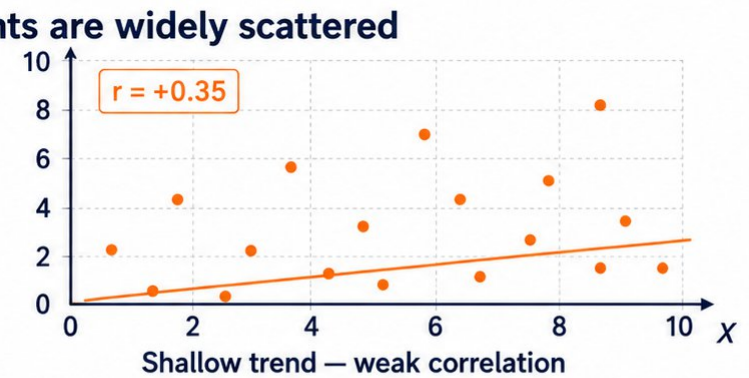
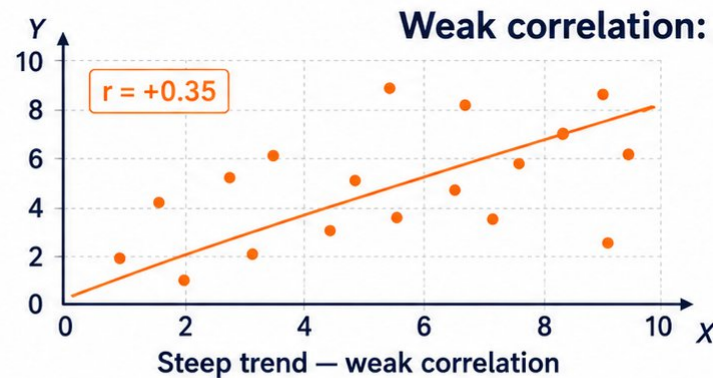
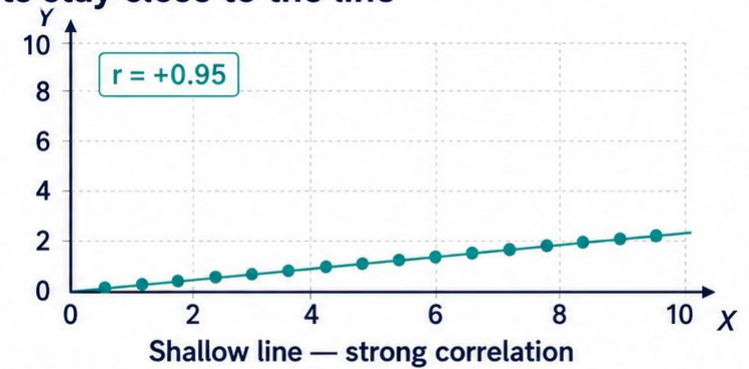
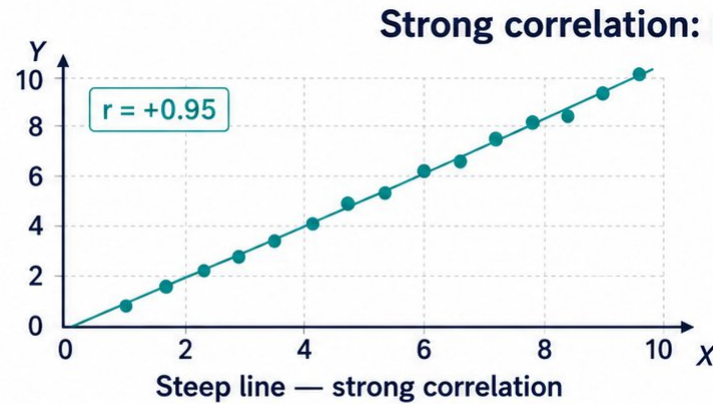
- Points close to a straight line → strong correlation
- Points widely scattered → weak correlation



Correlation is not slope

Two lines can have the **same r -value** but **different steepness** because **correlation** and **slope measure different things**.

- The slope shows how quickly Y changes as X changes; it does not show correlation strength.
- Correlation shows r shows the direction and strength of the linear relationship.



Correlation strength = how closely the points follow a line

Slope = how steep the line is

The Housing Dataset

```
housingPath = 'housingData.csv'
housing = pd.read_csv(housingPath)
housing = housing.query('sqft_living < 8000 and price < 1000000 and price > 0')
housing['has_basement'] = housing['sqft_basement'].apply(lambda x: True if x > 0 else False)
# Drop unnecessary columns from the 'housing' DataFrame
housing = housing.drop(columns=['date', 'street', 'city', 'statezip', 'country',
                                'sqft_lot', 'yr_renovated', 'sqft_basement'])
```

```
housing.info()
```

```
<class 'pandas.core.frame.DataFrame'>
Int64Index: 4207 entries, 0 to 4599
Data columns (total 11 columns):
#   Column          Non-Null Count  Dtype
---  -
0   price            4207 non-null   float64
1   bedrooms         4207 non-null   float64
2   bathrooms        4207 non-null   float64
3   sqft_living      4207 non-null   int64
4   floors           4207 non-null   float64
5   waterfront       4207 non-null   int64
6   view            4207 non-null   int64
7   condition        4207 non-null   int64
8   sqft_above       4207 non-null   int64
9   yr_built         4207 non-null   int64
10  has_basement     4207 non-null   bool
dtypes: bool(1), float64(4), int64(6)
memory usage: 365.6 KB
```

```
housing.head()
```

	price	bedrooms	bathrooms	sqft_living	floors	waterfront	view	condition	sqft_above	yr_built	has_basement
0	313000.0	3.0	1.50	1340	1.5	0	0	3	1340	1955	False
2	342000.0	3.0	2.00	1930	1.0	0	0	4	1930	1966	False
3	420000.0	3.0	2.25	2000	1.0	0	0	4	1000	1963	True
4	550000.0	4.0	2.50	1940	1.0	0	0	4	1140	1976	True
5	490000.0	2.0	1.00	880	1.0	0	0	3	880	1938	False

Identifying Correlations using r-values

```
# Calculate the correlation matrix using corr()
correlation_matrix = housing.corr()
correlation_matrix.head()
```

	price	bedrooms	bathrooms	sqft_living	floors	waterfront	view	condition	sqft_above	yr_built	has_basement
price	1.000000	0.305046	0.460155	0.607379	0.272151	0.073671	0.196460	0.044926	0.517395	0.053012	0.152591
bedrooms	0.305046	1.000000	0.516726	0.603655	0.148377	-0.032263	0.060074	0.015105	0.466683	0.154287	0.157001
bathrooms	0.460155	0.516726	1.000000	0.715626	0.495380	0.020700	0.112797	-0.145555	0.637045	0.521801	0.127437
sqft_living	0.607379	0.603655	0.715626	1.000000	0.325647	0.030473	0.184432	-0.086310	0.848312	0.331001	0.175223
floors	0.272151	0.148377	0.495380	0.325647	1.000000	0.008710	-0.006647	-0.298350	0.526227	0.502039	-0.305249

```
# Calculate the correlation matrix using corr()
# Select only the 'price' column using indexing and store it in a new DataFrame
# Sort the DataFrame by 'price' column in descending order using the sort_values() method
correlation_with_price = housing.corr()[['price']].sort_values(by='price', ascending=False)
correlation_with_price
```

	price
price	1.000000
sqft_living	0.607379
sqft_above	0.517395
bathrooms	0.460155
bedrooms	0.305046
floors	0.272151
view	0.196460
has_basement	0.152591
waterfront	0.073671
yr_built	0.053012
condition	0.044926

Correlation is not Causation

Correlation means **two variables are related or change together**.

Causation means **a change in one variable directly produces a change in another**.

Correlation does not prove causation means that when two variables change together, we cannot immediately conclude that one causes the other.

Ex:

More umbrellas are observed when more car accidents occur.

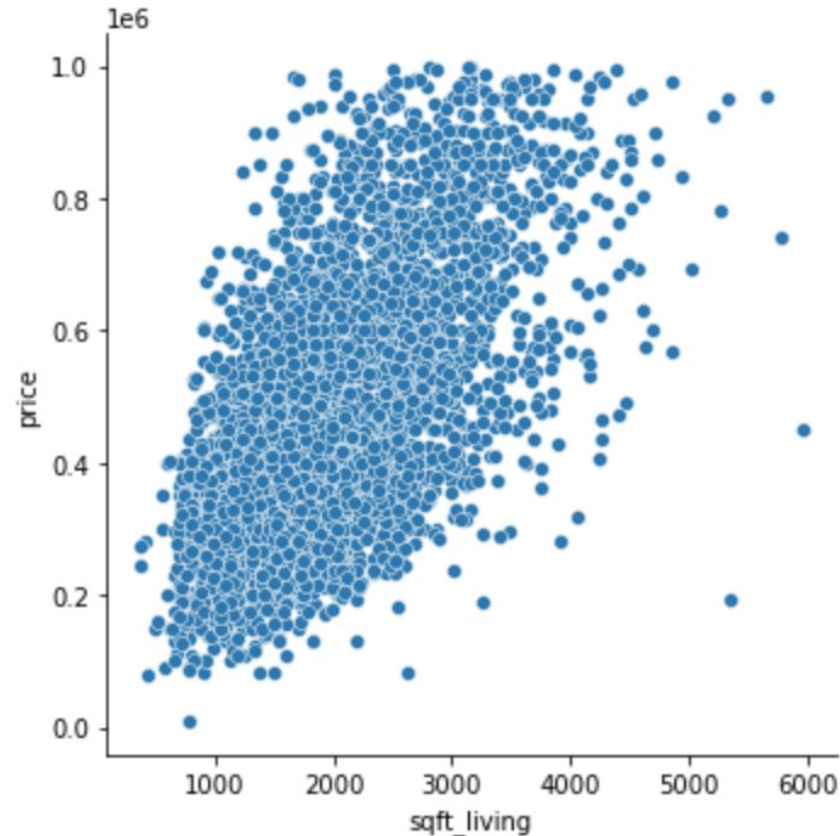
This does not mean umbrellas cause car accidents. **Rainy weather** is the confounding variable because it affects both:

Correlation	Causation
Variables change together	One variable directly affects another
Shows a relationship	Shows cause and effect
Can be coincidental or influenced by another variable	Requires stronger evidence, often from controlled experiments

Identifying Correlations using Scatterplot

```
# Create a scatter plot using seaborn's relplot
# The 'data' parameter is set to the 'housing' DataFrame
# The 'x' parameter is set to 'sqft_living' column and the 'y' parameter is set to 'price' column
sns.relplot(data=housing, x='sqft_living', y='price')
```

<seaborn.axisgrid.FacetGrid at 0x11ce8b460>

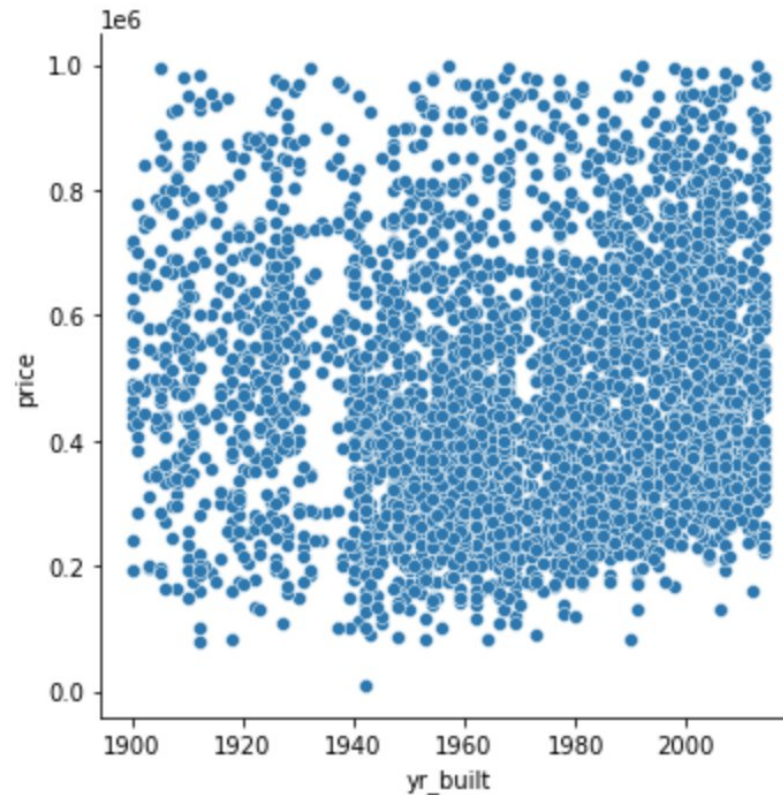


	price
price	1.000000
sqft_living	0.607379
sqft_above	0.517395
bathrooms	0.460155
bedrooms	0.305046
floors	0.272151
view	0.196460
has_basement	0.152591
waterfront	0.073671
yr_built	0.053012
condition	0.044926

Identifying Correlations using Scatterplot (Cont.)

```
# Create a scatter plot using seaborn's relplot
# The 'data' parameter is set to the 'housing' DataFrame
# The 'x' parameter is set to 'yr_built' column and the 'y' parameter is set to 'price' column
sns.relplot(data=housing, x='yr_built', y='price')
```

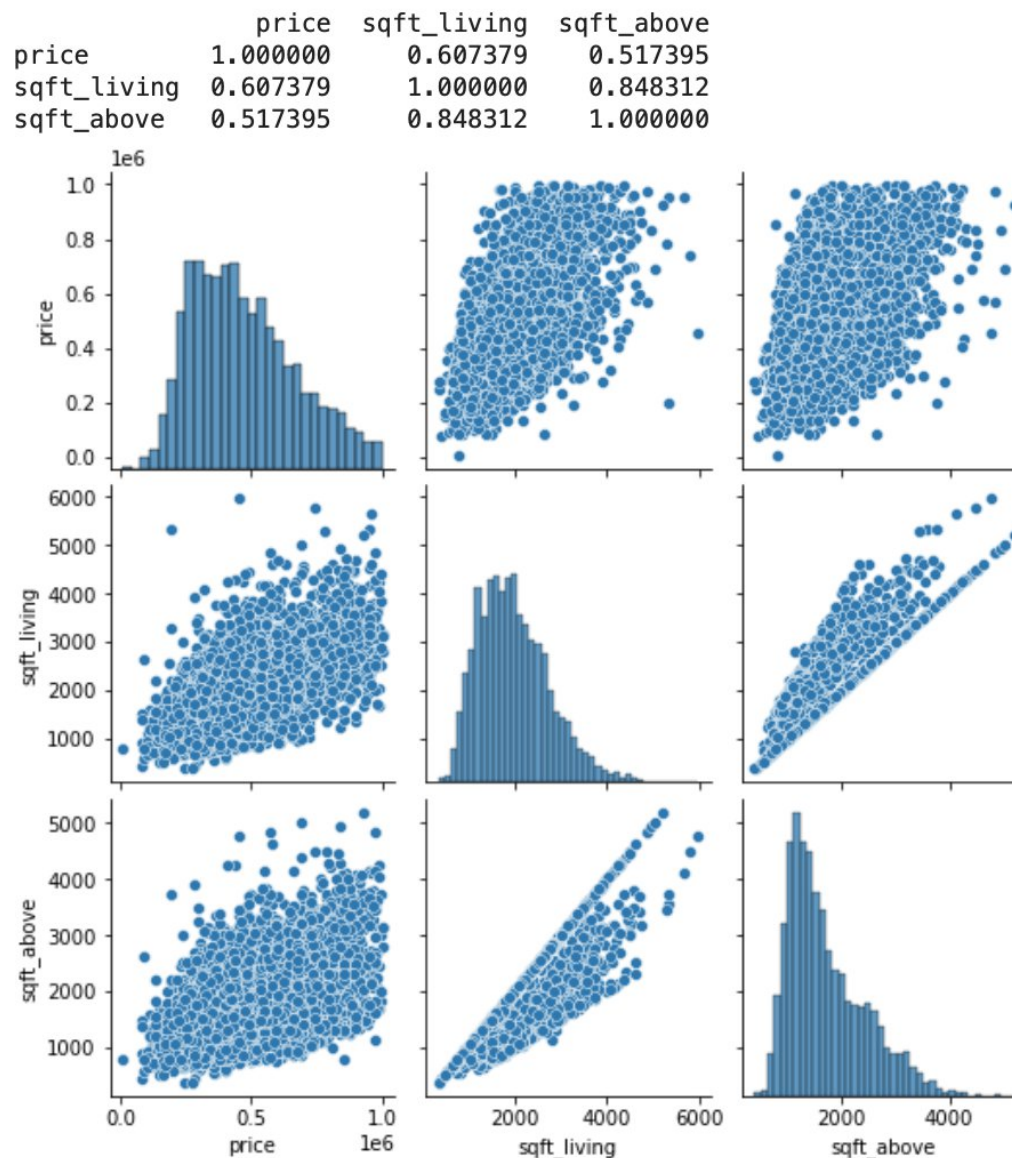
<seaborn.axisgrid.FacetGrid at 0x11ce8b730>



	price
price	1.000000
sqft_living	0.607379
sqft_above	0.517395
bathrooms	0.460155
bedrooms	0.305046
floors	0.272151
view	0.196460
has_basement	0.152591
waterfront	0.073671
yr_built	0.053012
condition	0.044926

Identifying Correlations using a Grid of Scatterplots (Cont.)

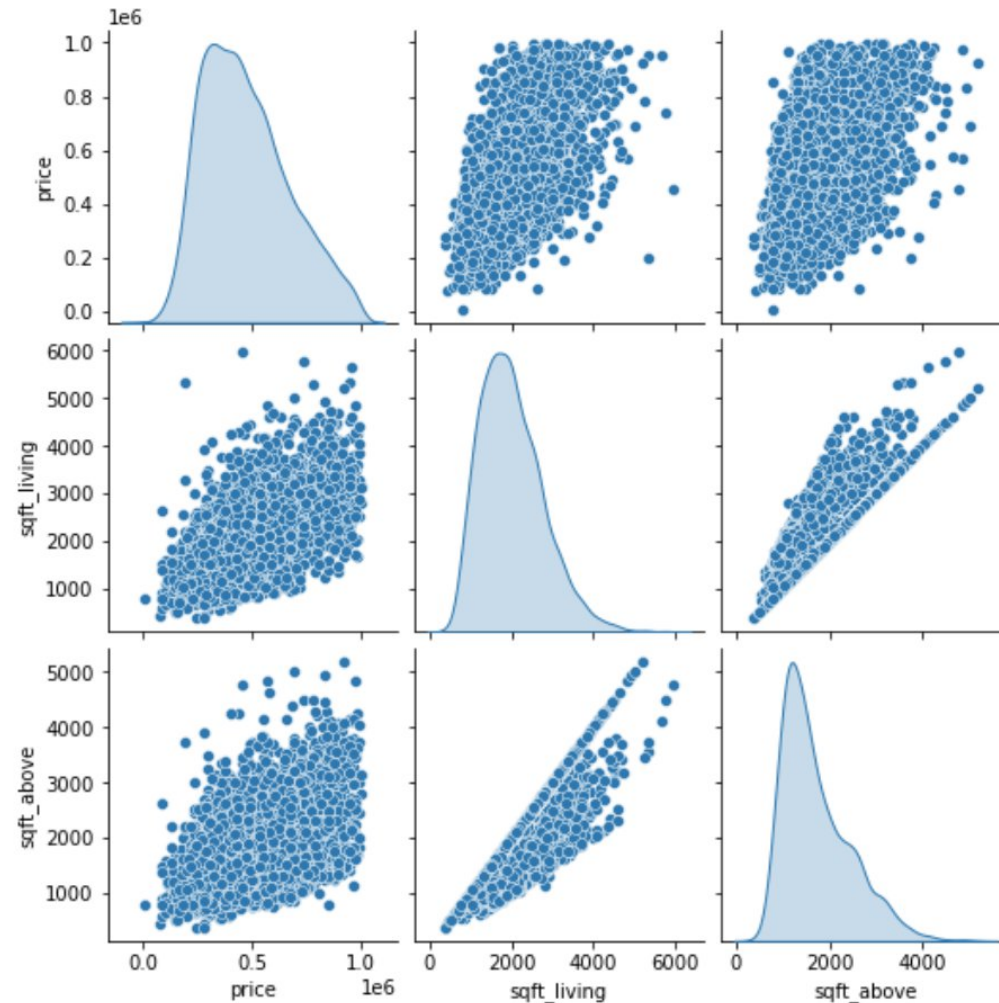
```
print(housing[['price', 'sqft_living', 'sqft_above']].corr())  
# Create a pairplot using seaborn's pairplot function  
# The 'x_vars' and 'y_vars' parameters are set to 'price', 'sqft_living', 'sqft_above'  
# The 'diag_kind' parameter is set to 'kde' to display kernel density estimates on the diagonal  
sns.pairplot(data=housing,  
             y_vars=['price', 'sqft_living', 'sqft_above'],  
             x_vars=['price', 'sqft_living', 'sqft_above']);
```



Identifying Correlations using a Grid of Scatterplots (Cont.)

```
print(housing[['price', 'sqft_living', 'sqft_above']].corr())  
# Create a pairplot using seaborn's pairplot function  
# The 'x_vars' and 'y_vars' parameters are set to 'price', 'sqft_living', 'sqft_above'  
# The 'diag_kind' parameter is set to 'kde' to display kernel density estimates on the diagonal  
sns.pairplot(data=housing,  
             y_vars=['price', 'sqft_living', 'sqft_above'],  
             x_vars=['price', 'sqft_living', 'sqft_above'],  
             diag_kind='kde');
```

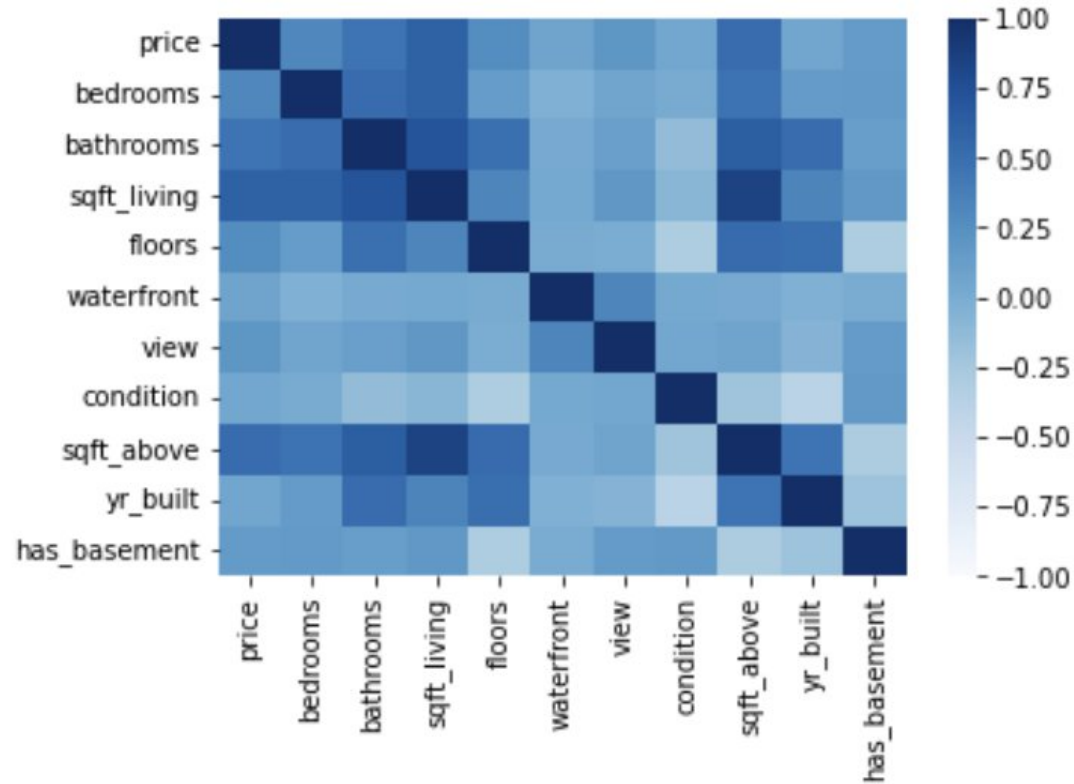
	price	sqft_living	sqft_above
price	1.000000	0.607379	0.517395
sqft_living	0.607379	1.000000	0.848312
sqft_above	0.517395	0.848312	1.000000



Identifying Correlations using a Heatmap

```
# Create a heatmap of the correlation matrix using the sns.heatmap() function
# Pass the correlation matrix as the data argument
# Use the 'Blues' colormap for the heatmap
# Set the minimum and maximum values for the color scale using vmin and vmax parameters
sns.heatmap(data=housing.corr(), cmap='Blues', vmin=-1.0, vmax=1.0)
```

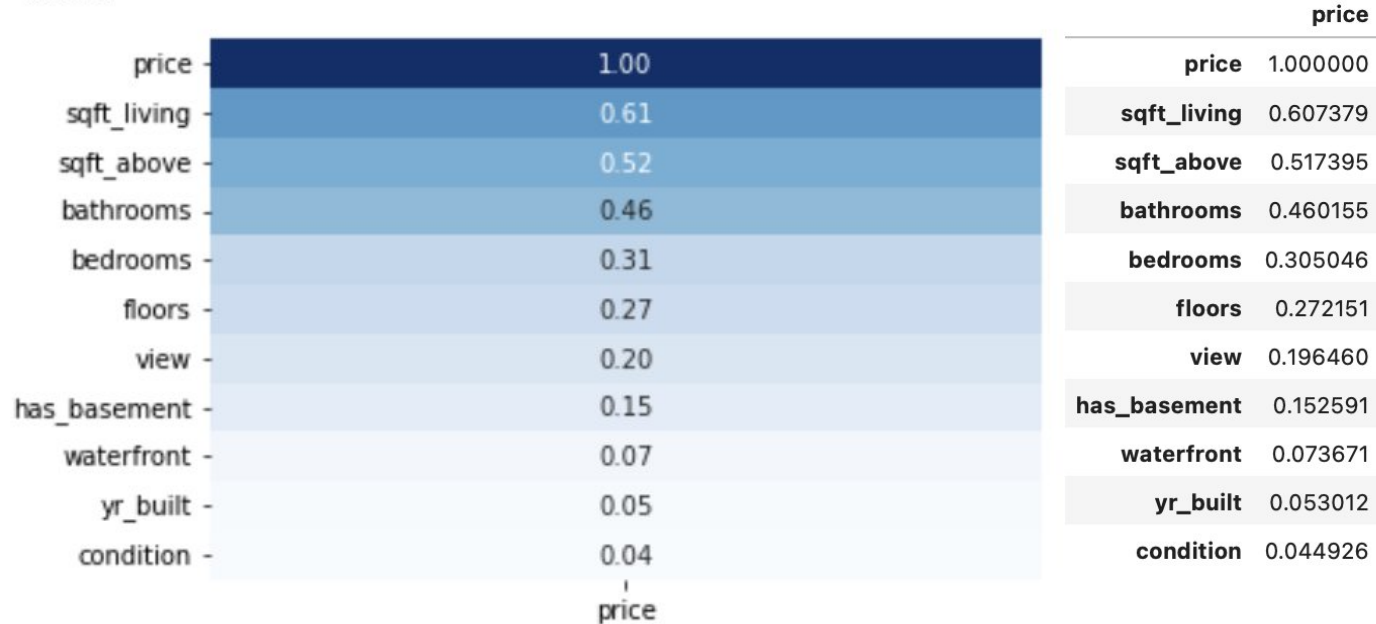
<Axes: >

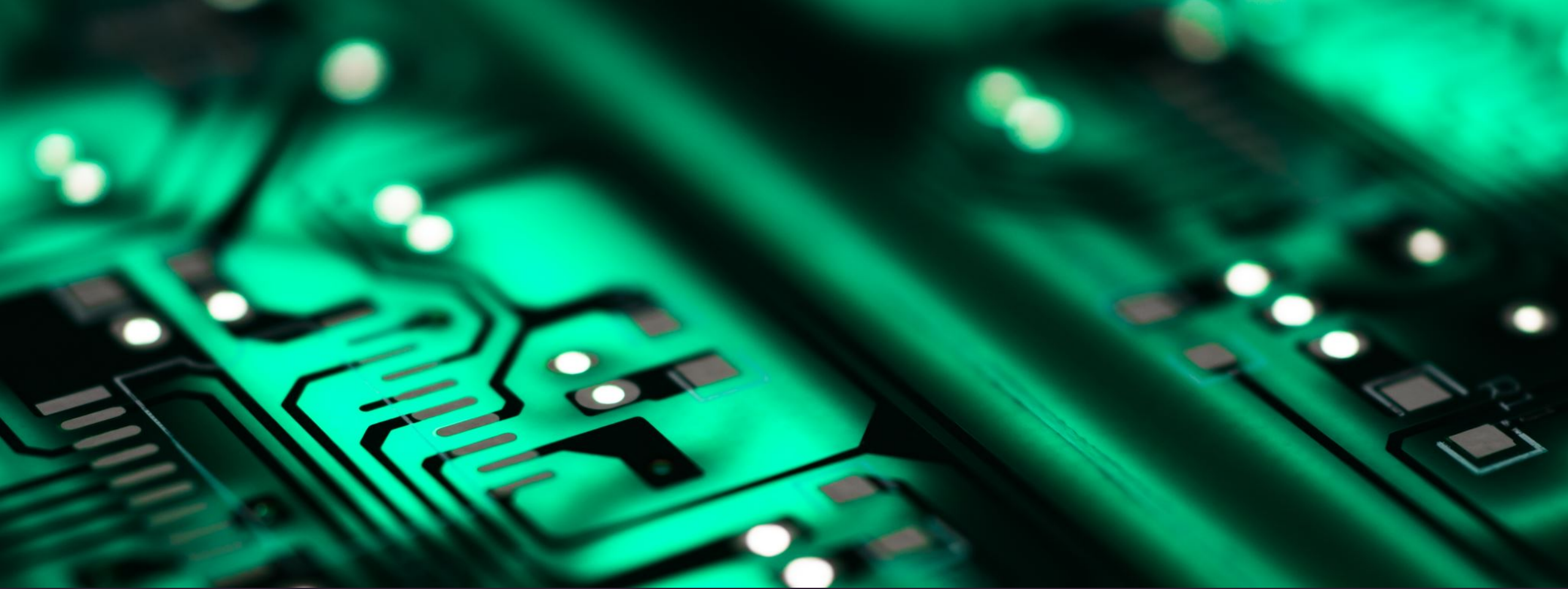


Identifying Correlations using a Heatmap (Cont.)

```
# Create a heatmap of the correlation matrix of 'housing' DataFrame with only the 'price' column
# Sort the correlations by the 'price' column in descending order
# Use the sns.heatmap() function with the specified parameters:
# - Pass the correlation matrix with only the 'price' column as the data argument
# - Set annot=True to show the correlation values as annotations
# - Use the 'Blues' colormap for the heatmap
# - Set cbar=False to hide the color bar
# - Use fmt='.2f' to format the annotations as floating-point numbers with 2 decimal places
sns.heatmap(data=housing.corr()[['price']].sort_values(by='price', ascending=False),
            annot=True, cmap='Blues', cbar=False, fmt='.2f')
```

<Axes: >





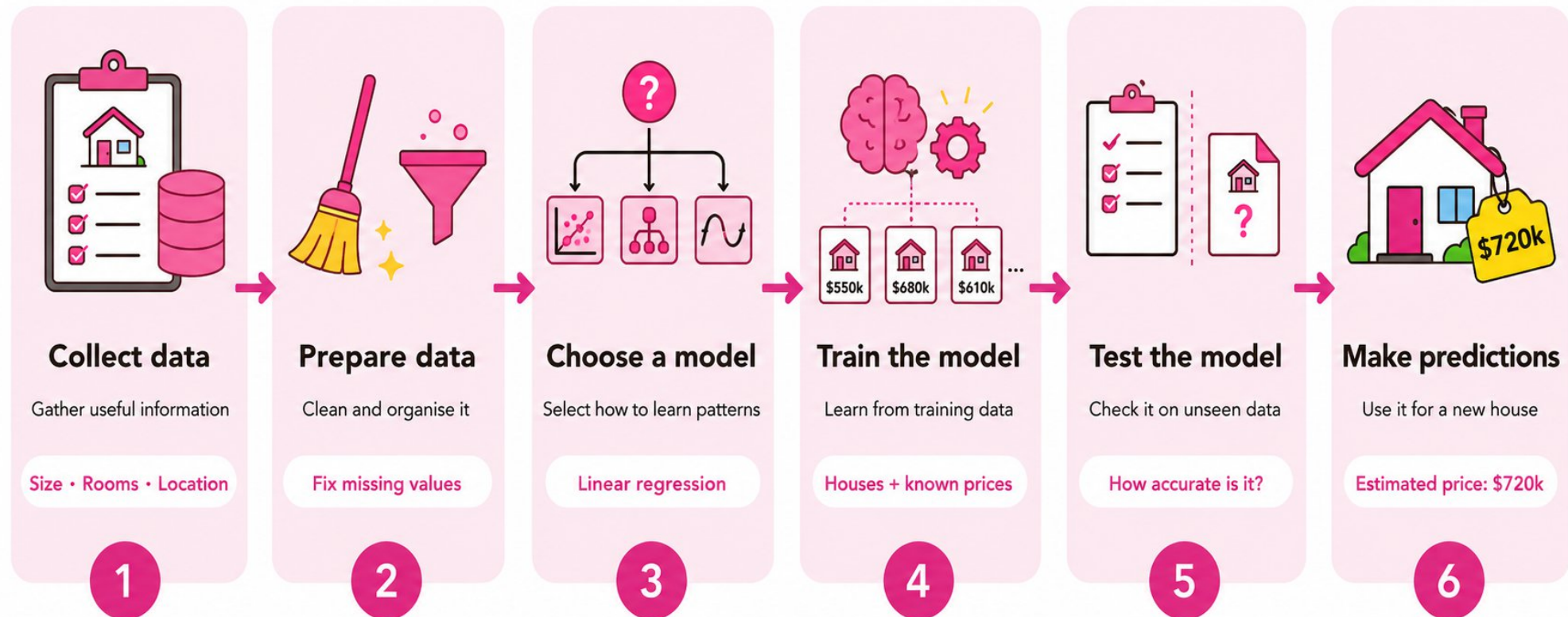
Predictive Models

- Statistical or machine learning models that use data to make predictions.
- Designed to identify patterns in the data and use those patterns to predict outcomes.
- Can be used in a wide range of applications, including customer relationship management, fraud detection, risk assessment, marketing, and healthcare.
- Used in businesses and organizations to improve decision-making and optimize performance.



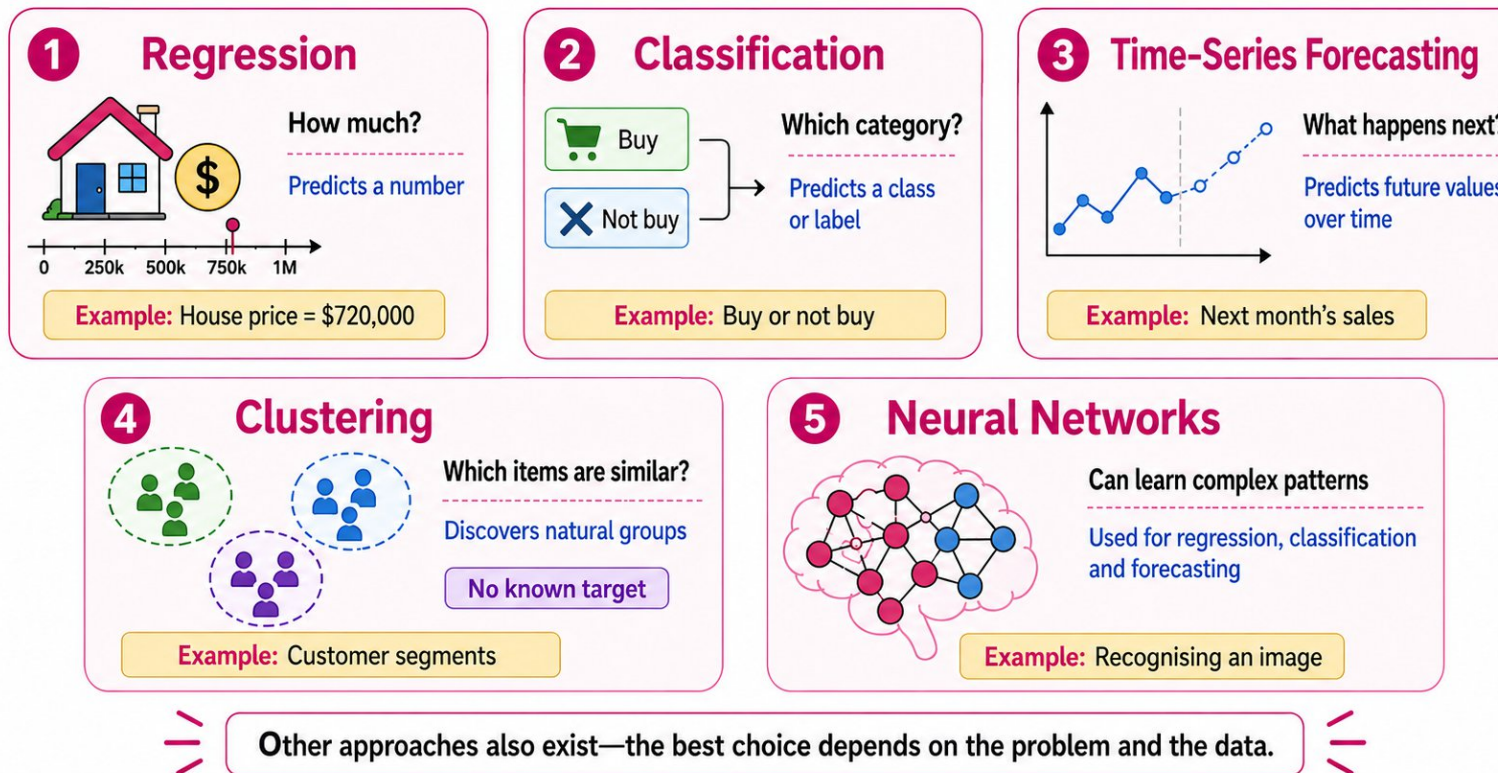
What are Predictive Models?

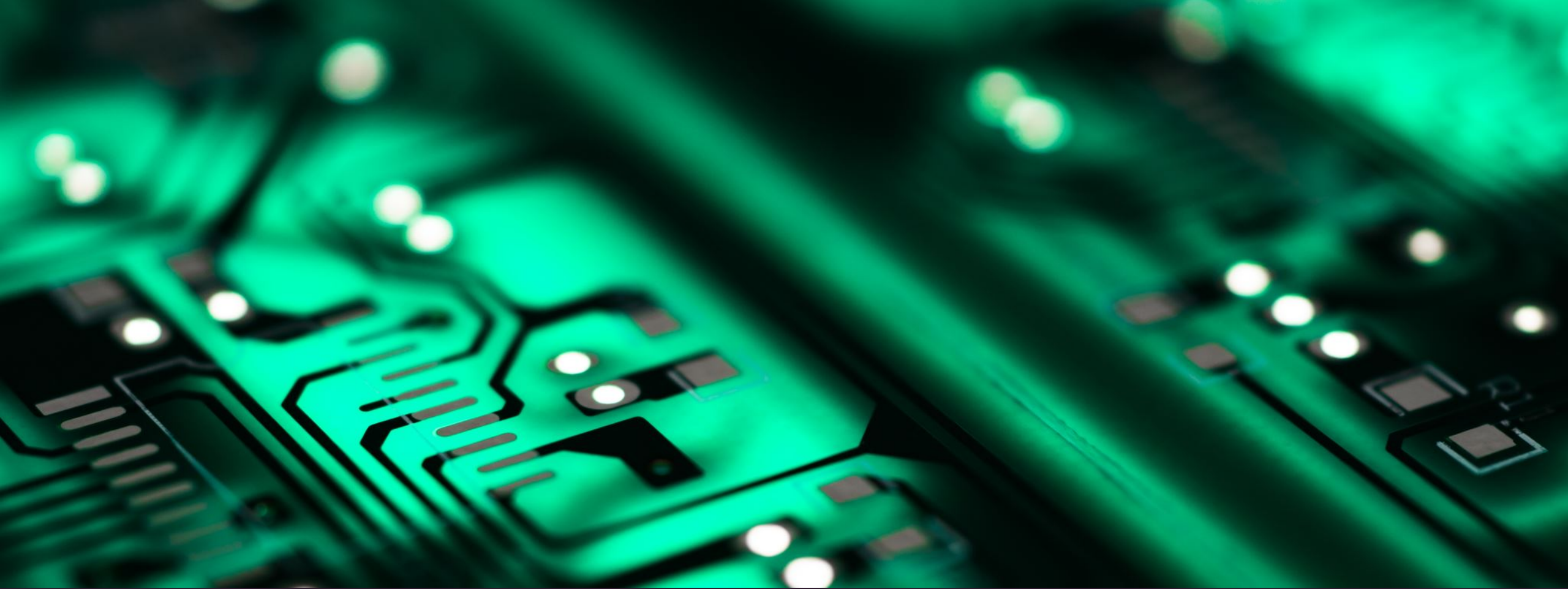
Building Predictive Models



Types of Predictive Models

Predictive models can come in many different forms, depending on the **problem** being solved and the **nature of the data**.



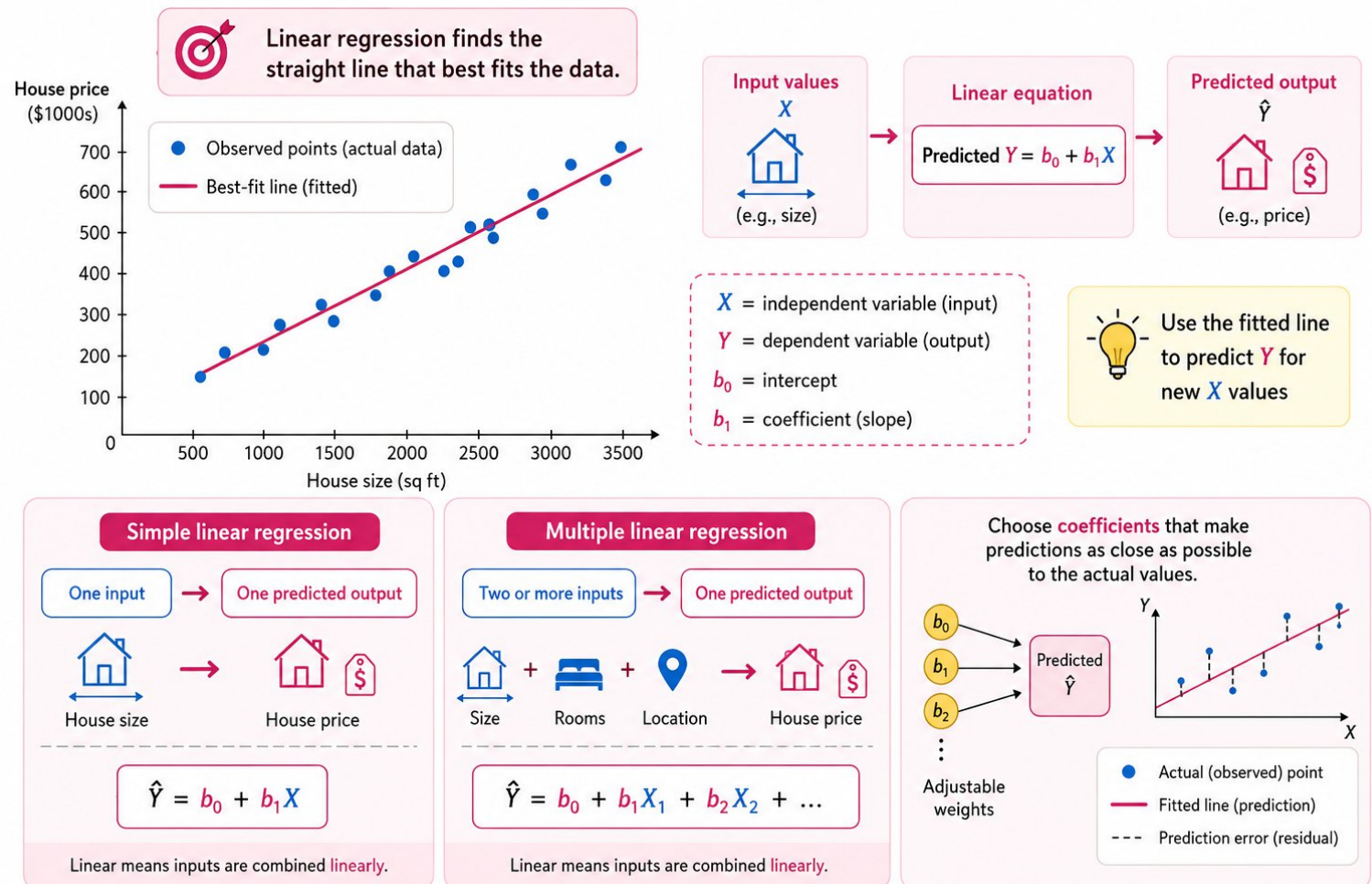


Linear Regression Models

Linear Regression

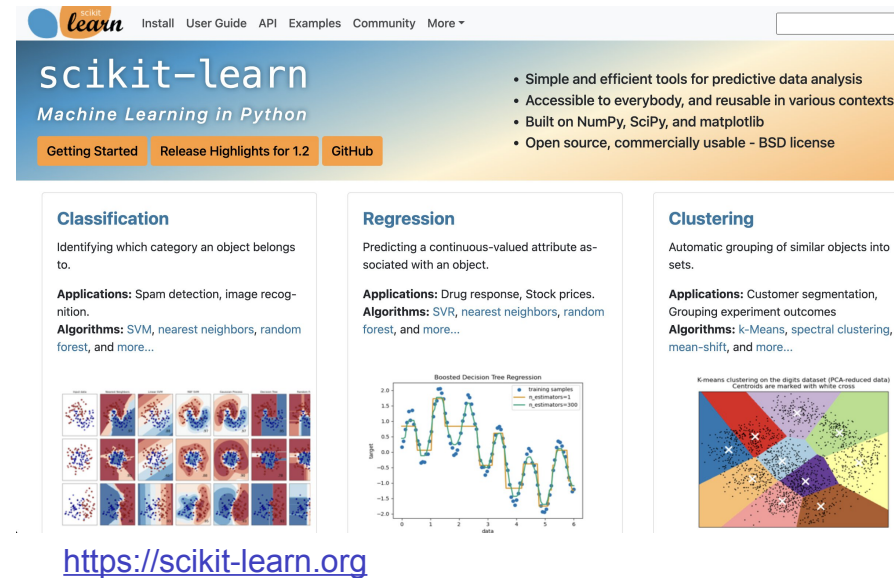
Linear regression assumes a linear relationship between the variables; the dependent variable can be expressed as a linear combination of the independent variables.

- A simple linear regression uses only one independent variable,
- A multiple linear regression uses two or more independent variables.



The sklearn (scikit-learn) library

- The `sklearn` (scikit-learn) library is a popular machine learning library in Python that provides a wide range of **tools for building predictive models**.
- Instead of programming a machine-learning algorithm from scratch, you can use its existing models and functions



The screenshot shows the scikit-learn website homepage. At the top, there's a navigation bar with links: Install, User Guide, API, Examples, Community, and More. The main header features the 'scikit-learn' logo and the tagline 'Machine Learning in Python'. Below this, there are three buttons: 'Getting Started', 'Release Highlights for 1.2', and 'GitHub'. To the right, a list of features is displayed: 'Simple and efficient tools for predictive data analysis', 'Accessible to everybody, and reusable in various contexts', 'Built on NumPy, SciPy, and matplotlib', and 'Open source, commercially usable - BSD license'. The page is divided into three main sections: 'Classification' (describing object categorization with applications like spam detection and algorithms like SVM), 'Regression' (describing continuous value prediction with applications like drug response and algorithms like SVR), and 'Clustering' (describing automatic grouping of objects with applications like customer segmentation and algorithms like k-Means). Each section includes a small representative figure.

<https://scikit-learn.org>

LinearRegression() creates the **model**.

.fit() **trains** it using known examples.

.predict() uses the learned relationship to predict new outcomes.

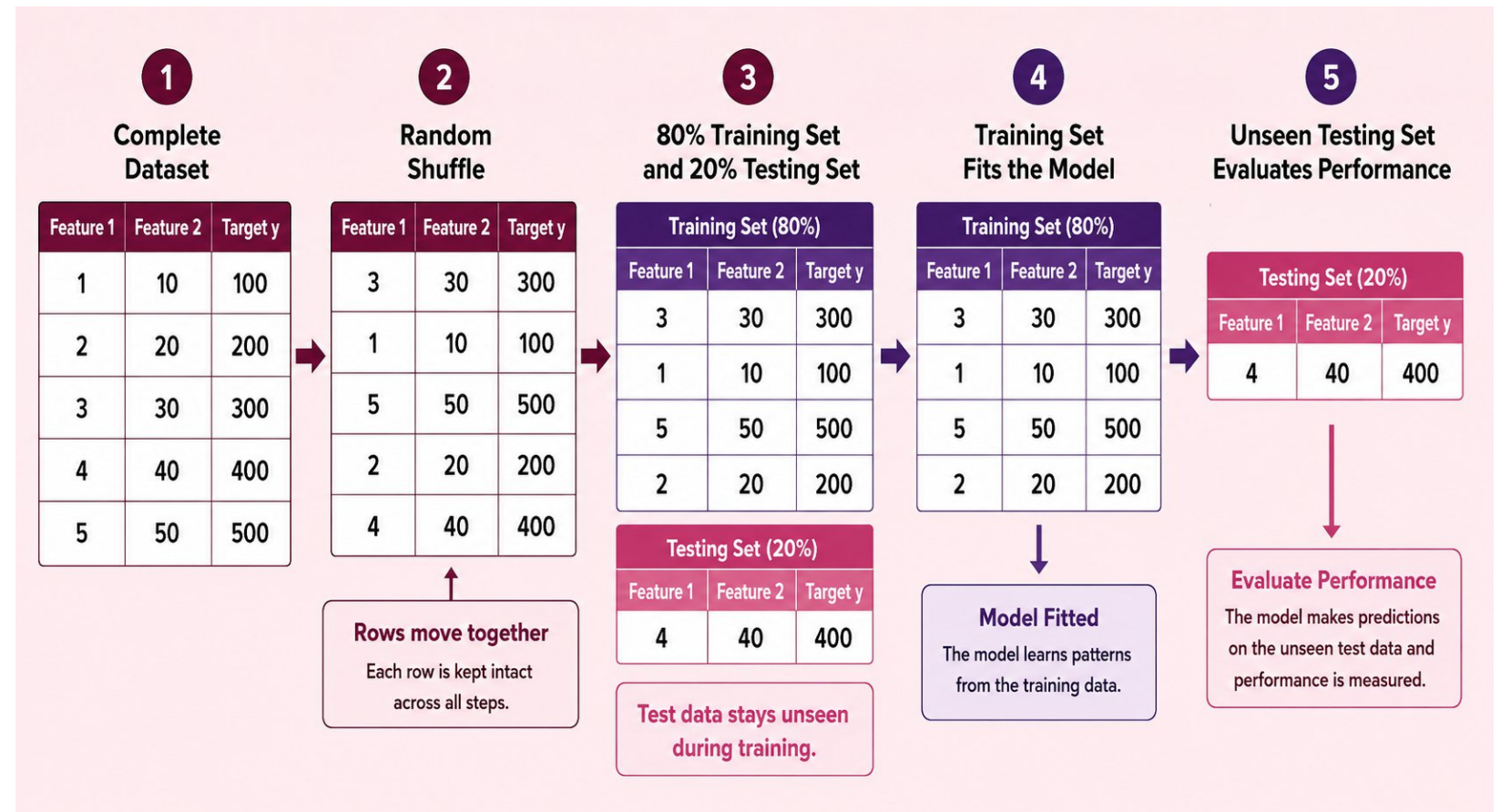
```
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
```

The `train\_test\_split` Function

To split a dataset into two subsets: a training set and a testing set.

- The training set is used to **train a model**,
- The testing set is used to **evaluate the model's performance** on unseen data.



The `train\_test\_split` function

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42
)
```

- **X**: input features, such as house size and number of rooms.
- **y**: target variable, such as house price.
- **test_size=0.2**: reserves 20% of the data for testing; the remaining 80% is used for training.
- **random_state=42**: ensures that the same rows are selected each time the code runs.

The model-building process is:

```
model.fit(X_train, y_train)

predictions = model.predict(X_test)
```

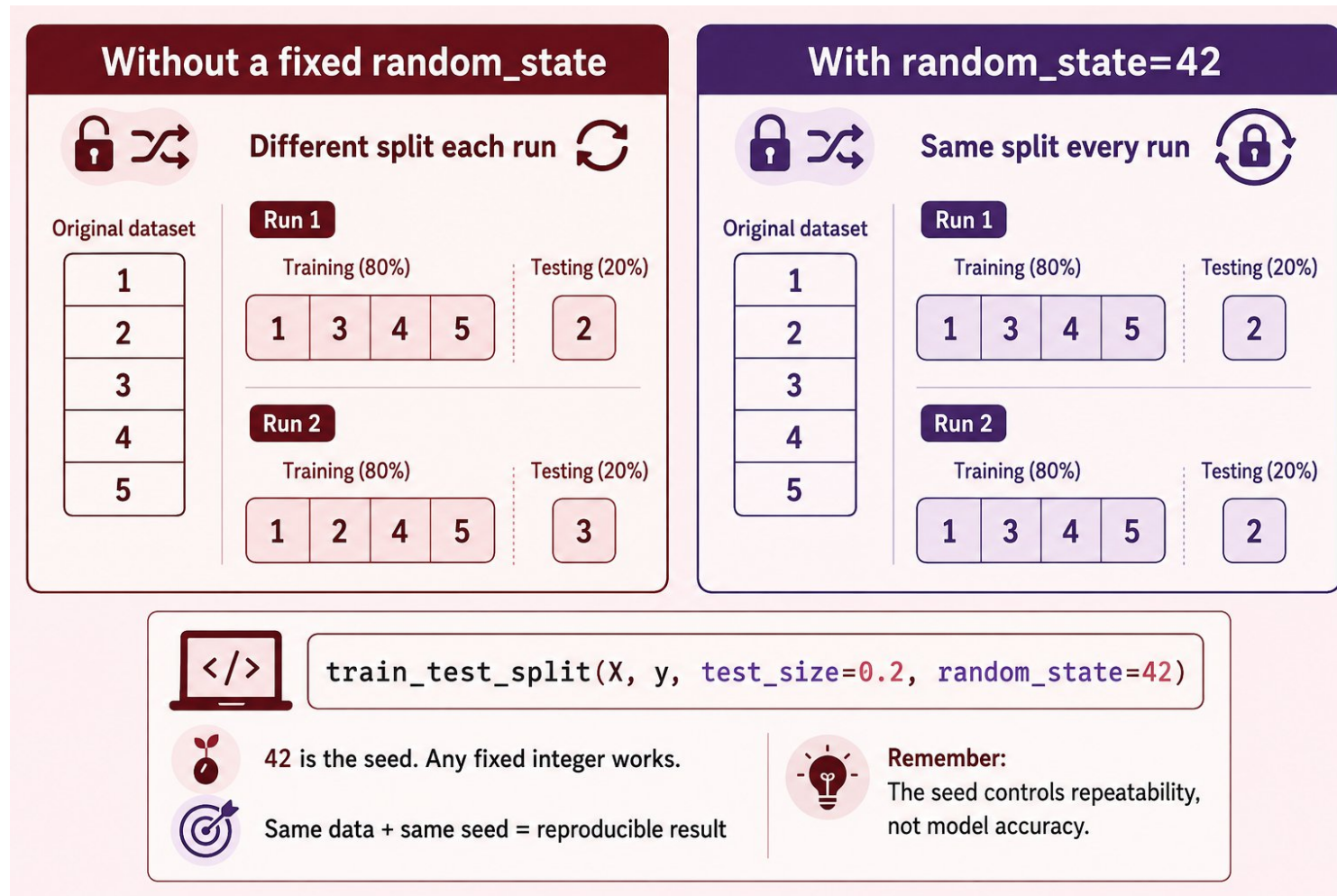
The predictions are then compared with y_test:

```
score = model.score(X_test, y_test)
```

Make predictions first and then evaluate them.

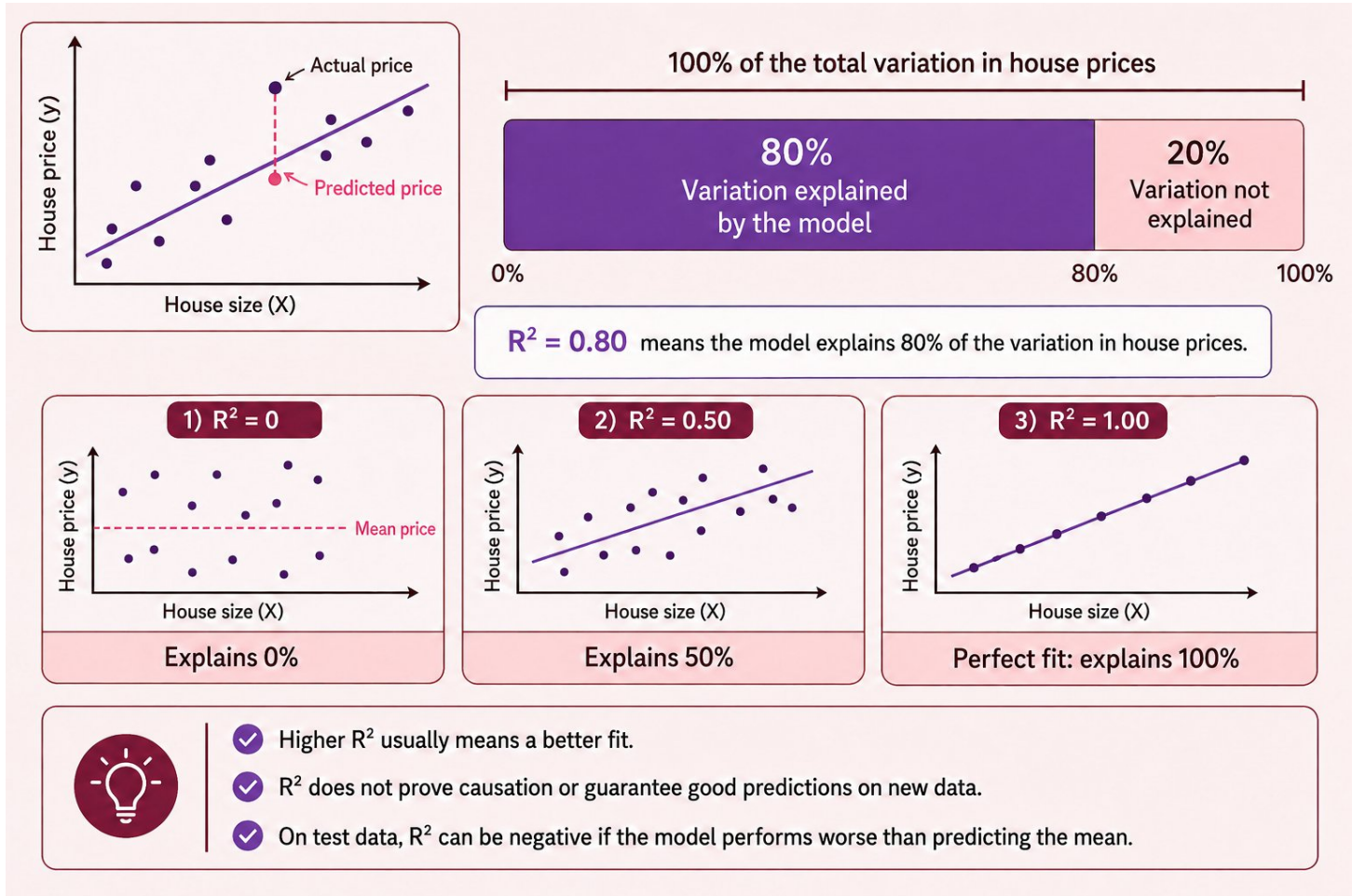
Setting a Random Seed

- Many algorithms make use of randomness, such as initialising weights, shuffling data, or splitting datasets.
- Using the same seed produces the same random result each time the code runs. (reproducible)
- 42 is an arbitrary fixed integer; another integer could also be used.
- Same data + same seed = same train/test split.
- Seed controls repeatability; it does not automatically improve model accuracy.



The R-squared Score; Coefficient of determination

- It is a statistical measure that represents the proportion of the variance in the dependent variable that is explained by the independent variables in a linear regression model.
- R^2 compares the regression model with a simple baseline that always predicts the mean.
- If the regression model makes much smaller errors than the baseline, R^2 is close to 1.
- The R-squared score ranges from 0 to 1, with higher values indicating a better fit of the model to the data.



Can we trust R-squared?

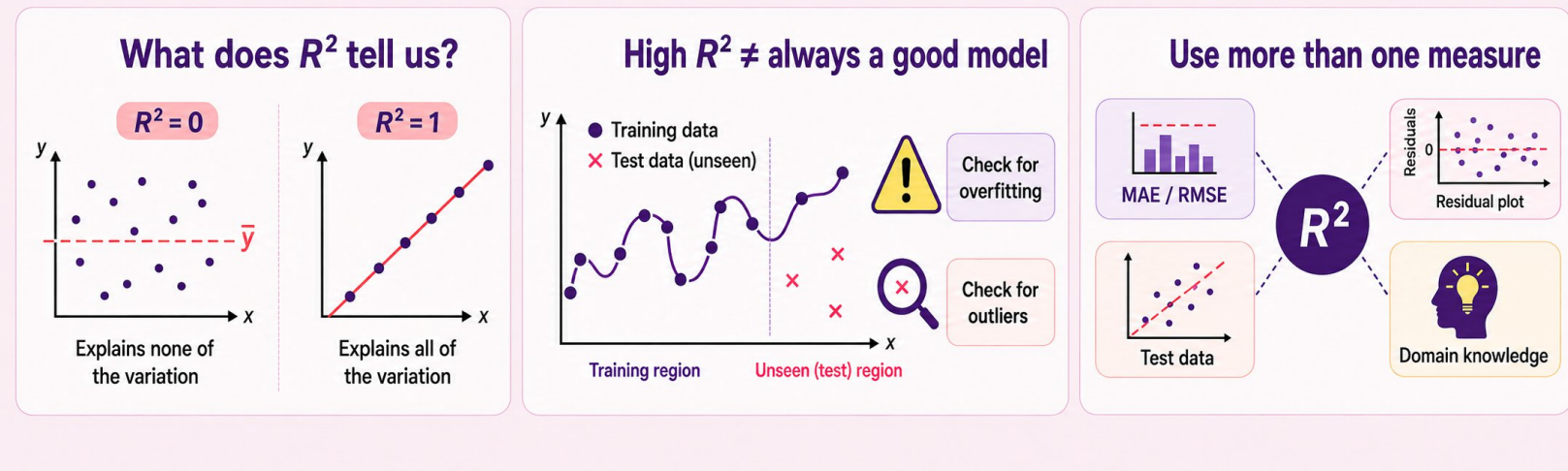
A **high R^2** means the model explains a large proportion of the variation in the data, but it **does not guarantee** that the **model is reliable**.

Overfitting: The model may learn the **training** data **too closely**, including its noise. It can achieve a high R^2 on the training data but perform **poorly on new, unseen data**.

Outliers: A few **extreme data points** can strongly **influence** the fitted regression line and the **R^2 value**. The score may appear high even though the model does not represent most observations well.

Therefore, always check:

- R^2 on **unseen test data**
- The **scatterplot and residual plot**
- Error measures such as **MAE** or **RMSE** **influential outliers**
- Whether are present

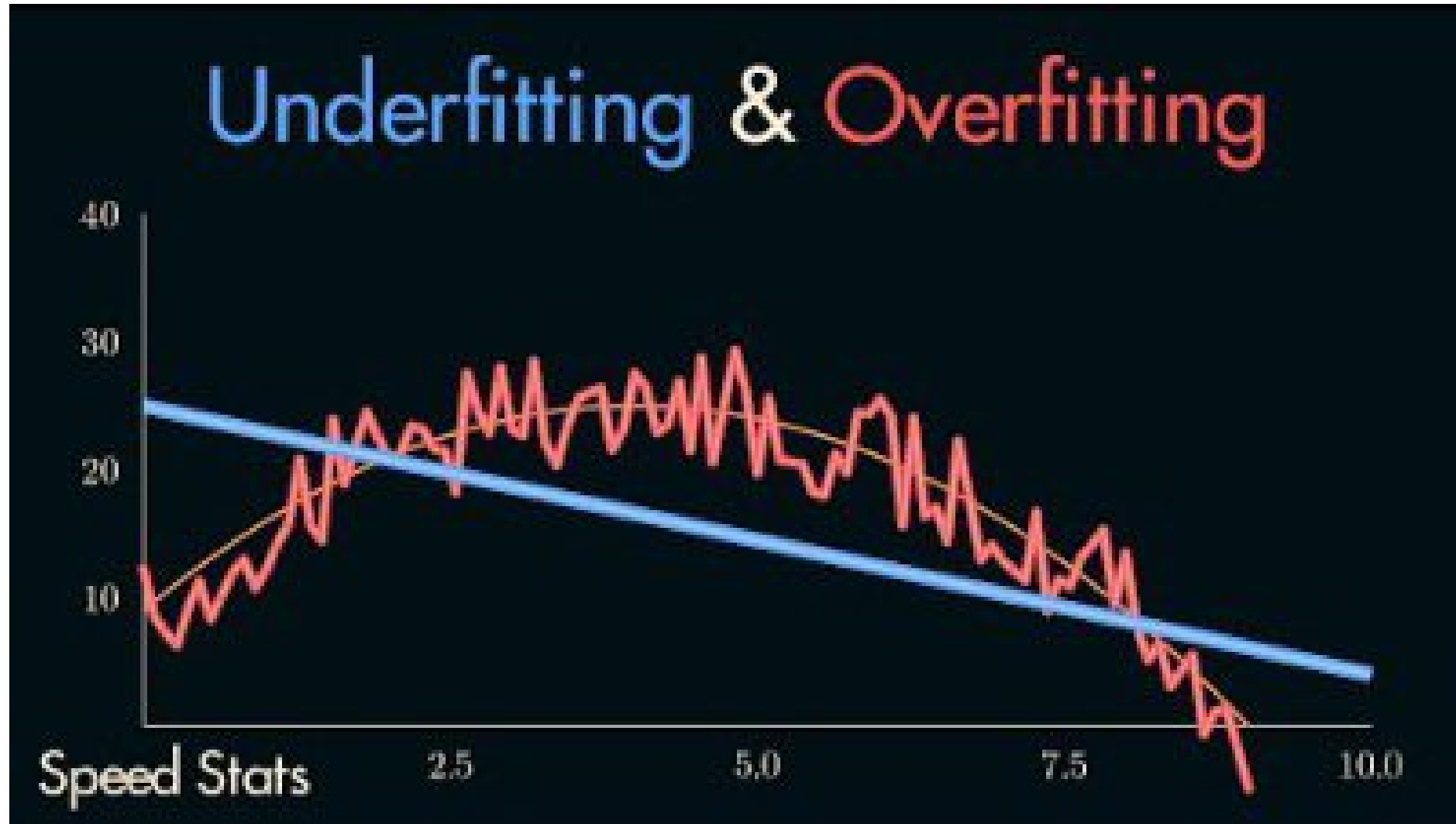


Movie Time 🎬



3-Minute Data Science

Movie Time 🌴



Linear Regression Model to Make Predictions

```
# Use the linear model to make predictions
y_predicted = linearModel.predict(x_test)
# Combine the x_test, y_test, and y_predicted values into a single dataframe
df = pd.concat([x_test.reset_index(drop=True), y_test.reset_index(drop=True),
                pd.DataFrame(y_predicted)], axis=1)
df.columns = ['sqft_living', 'price', 'price_predicted']
display(df.head())
```

	sqft_living	price	price_predicted
0	1240	390000.0	355333.117428
1	2450	345000.0	545363.476872
2	1950	375000.0	466838.534953
3	2280	500324.0	518664.996619
4	2410	600000.0	539081.481518

```
# The function train_test_split is used to split the data into training and test sets.
# In this case, we use the housing data frame to split the data,
# selecting the 'sqft_living' column as the feature and the 'price' column as the target.
# test_size=0.33 means that 33% of the data will be used for testing and 67% for training.
# The random_state parameter sets the random seed for reproducibility.
# The resulting four variables represent the feature and target splits for both training and testing data.
x_train, x_test, y_train, y_test = train_test_split(housing[['sqft_living']],
                                                    housing[['price']], test_size=0.33, random_state=42)

# Create the linear regression model from the training dataset
# In this line, we create an instance of the LinearRegression class, which represents the linear regression model.
linearModel = LinearRegression()
# We use the fit() method of the LinearRegression instance to train the model using the training data.
# The fit() method takes the feature (x_train) and target (y_train) data as arguments.
# Once the model is trained, it will be able to predict the target values for new feature data.
linearModel.fit(x_train, y_train)

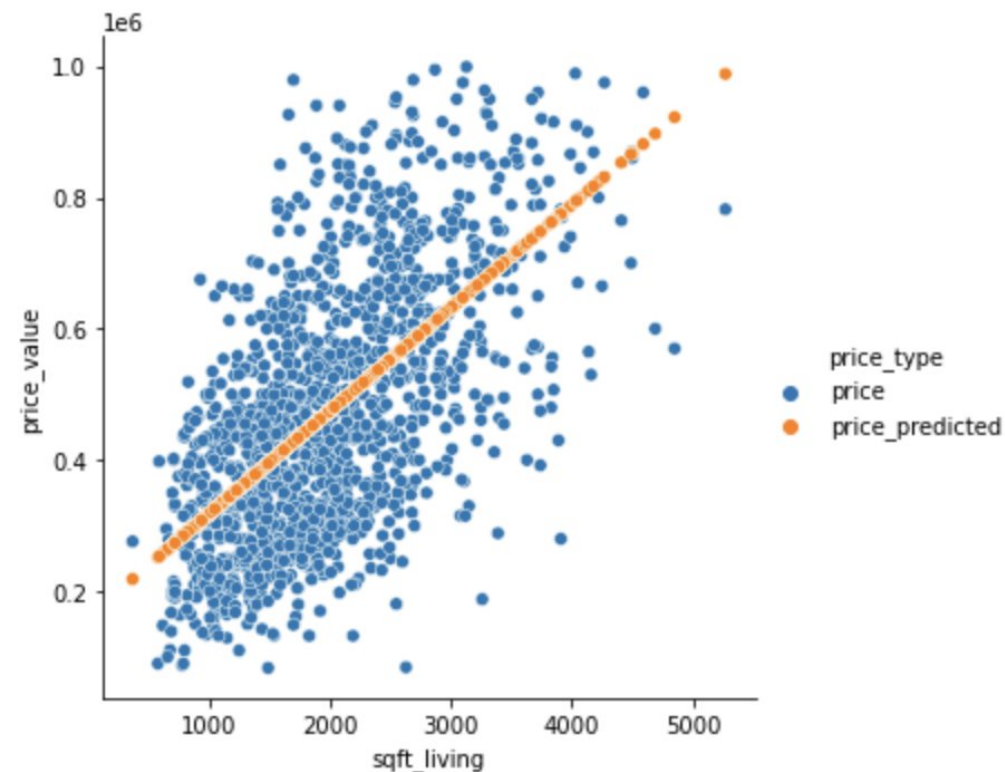
# Validate the linear model with the test dataset
# The score() method of the LinearRegression instance is used to validate the model using the test data.
# The score() method takes the feature (x_test) and target (y_test) data as arguments.
# It returns the R-squared score of the model, which indicates how well the model fits the data.
linearModel.score(x_test, y_test)
```

0.3507265465332233

Plotting the Predicted Data

```
# Create a DataFrame to hold the predicted values
predicted = pd.DataFrame(y_predicted, columns=['price_predicted'])
# Combine the test data and the predicted data
combined = predicted.join([x_test.reset_index(drop=True),
                           y_test.reset_index(drop=True)])
# Melt price and price_predicted columns into a single column
melted = pd.melt(combined, id_vars=['sqft_living'],
                  value_vars=['price', 'price_predicted'],
                  var_name='price_type', value_name='price_value')
display(melted)
# Create a scatter plot using Seaborn to visualize the melted data
sns.relplot(data=melted, x='sqft_living', y='price_value', hue='price_type');
```

	sqft_living	price_type	price_value
0	1240	price	390000.000000
1	2450	price	345000.000000
2	1950	price	375000.000000
3	2280	price	500324.000000
4	2410	price	600000.000000
...	...	...	...
2773	2280	price_predicted	518664.996619
2774	940	price_predicted	308218.152277
2775	1050	price_predicted	325493.639499
2776	1620	price_predicted	415012.073286
2777	1490	price_predicted	394595.588387



sns.lmplot()

sns.lmplot() is a Seaborn function that creates a **scatterplot with a fitted regression line**.

This displays:

- Each observation as a point.
- A regression line showing the estimated linear relationship.
- A shaded confidence interval around the regression line.

seaborn.lmplot

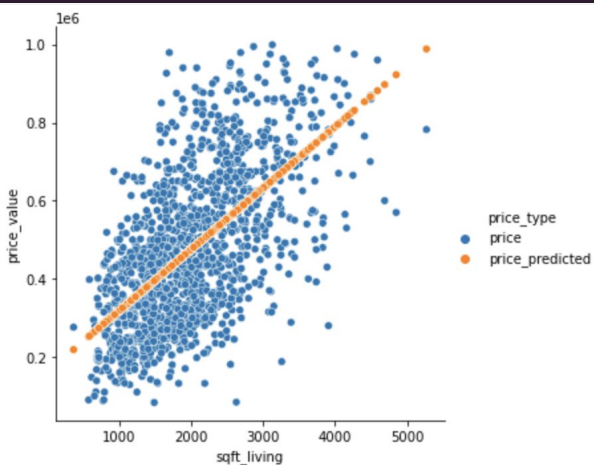
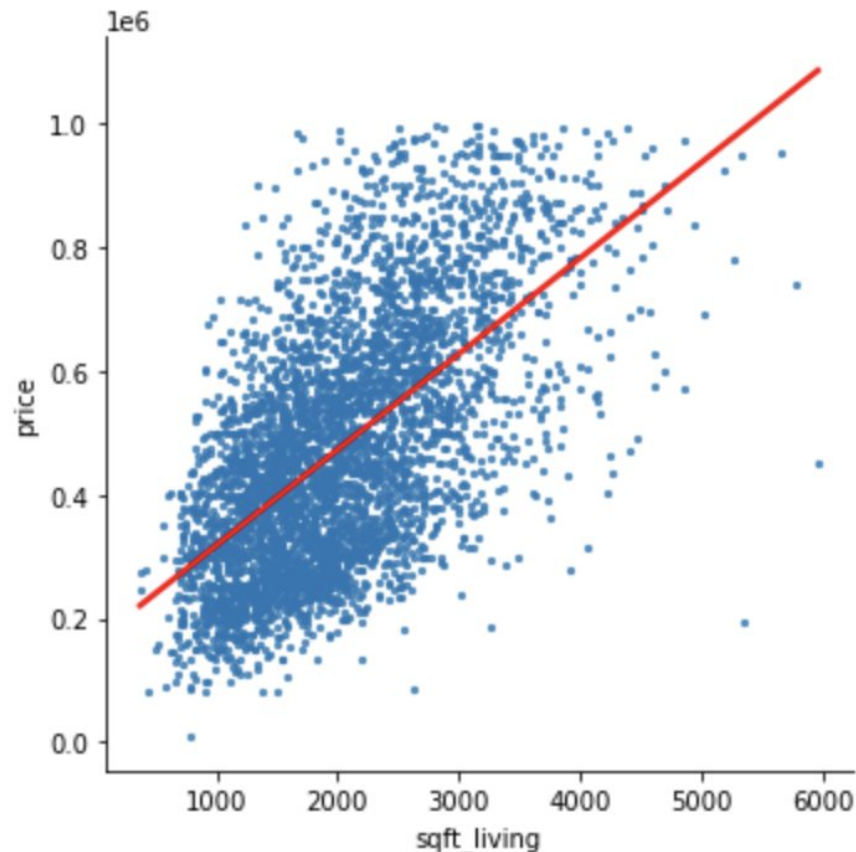
```
seaborn.lmplot(data, *, x=None, y=None, hue=None, col=None, row=None, palette=None,
col_wrap=None, height=5, aspect=1, markers='o', sharex=None, sharey=None, hue_order=None,
col_order=None, row_order=None, legend=True, legend_out=None, x_estimator=None, x_bins=None,
x_ci='ci', scatter=True, fit_reg=True, ci=95, n_boot=1000, units=None, seed=None, order=1,
logistic=False, lowess=False, robust=False, logx=False, x_partial=None, y_partial=None,
truncate=True, x_jitter=None, y_jitter=None, scatter_kws=None, line_kws=None,
facet_kws=None)
```

<https://seaborn.pydata.org/generated/seaborn.lmplot.html>

Plotting a Linear Regression

```
# Create a scatter plot with a red regression line using Seaborn
# The scatter plot displays the individual data points as dots on the plot,
# with 'sqft_living' values on the x-axis and 'price' values on the y-axis.
# The regression line is overlaid on the scatter plot as a line that best fits the data points.
# This line can be used to make predictions about the 'price' for any given 'sqft_living' value.
```

```
sns.lmplot(data=housing,          # The dataset to use
            x='sqft_living',      # The column to use for the x-axis
            y='price',            # The column to use for the y-axis
            ci=None,              # The size of the confidence interval for the regression line
            scatter_kws={'s':5},  # Additional scatter plot settings (size of markers)
            line_kws={'color':'red'}); # Additional regression line settings (color)
```



Residual

A **residual** is the difference between an observed actual value and the value predicted by a regression model.

$$\text{Residual} = y - \hat{y}$$

where: y is the actual value.

$\hat{y}$ is the predicted value.

On a regression scatterplot, a residual is the **vertical distance** between an observed data point and the fitted regression line at the same x -value:

Example

Suppose a house's actual price is \$750,000, but the model predicts \$720,000:

$$\text{Residual} = 750,000 - 720,000 = 30,000$$

The residual is +\$30,000, meaning the model underestimated the price.

If the model predicts \$780,000:

$$\text{Residual} = 750,000 - 780,000 = -30,000$$

The residual is -\$30,000, meaning the model overestimated the price.

Interpret Residual plot

Random pattern around zero → linear model may be appropriate

Curved pattern → nonlinear model may be needed

Funnel shape → error variance changes across x

Extreme isolated points → possible outliers

Residual above zero: actual price is higher than predicted, so the model underestimated.

Residual below zero: actual price is lower than predicted, so the model overestimated.

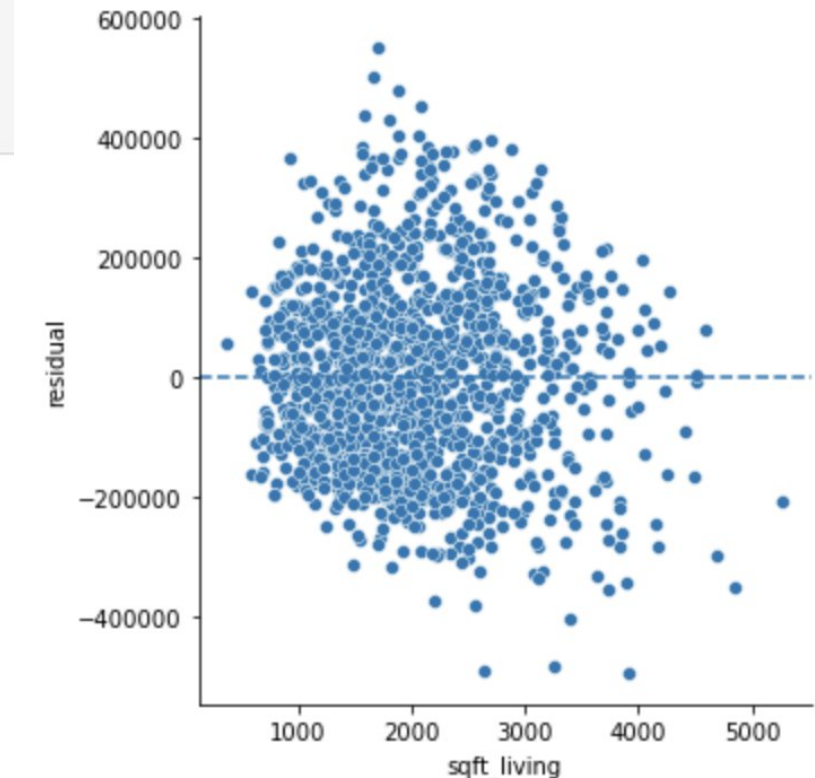
Residual near zero: the prediction is close to the actual price.

A good residual plot does not mean that every residual is zero. It means the residuals show no clear systematic pattern around zero.

Plotting the Residuals

```
# Calculate residuals and add them as a new column to the combined DataFrame
combined['residual'] = combined.price - combined.price_predicted
display(combined.head())
# Create a scatter plot using Seaborn to visualize the combined data with residuals
g = sns.relplot(data=combined, x='sqft_living', y='residual')
# Draw a horizontal line where the Y axis is 0
for ax in g.axes.flat:
    ax.axhline(0, ls='--')
```

	price_predicted	sqft_living	price	residual
0	355333.117428	1240	390000.0	34666.882572
1	545363.476872	2450	345000.0	-200363.476872
2	466838.534953	1950	375000.0	-91838.534953
3	518664.996619	2280	500324.0	-18340.996619
4	539081.481518	2410	600000.0	60918.518482



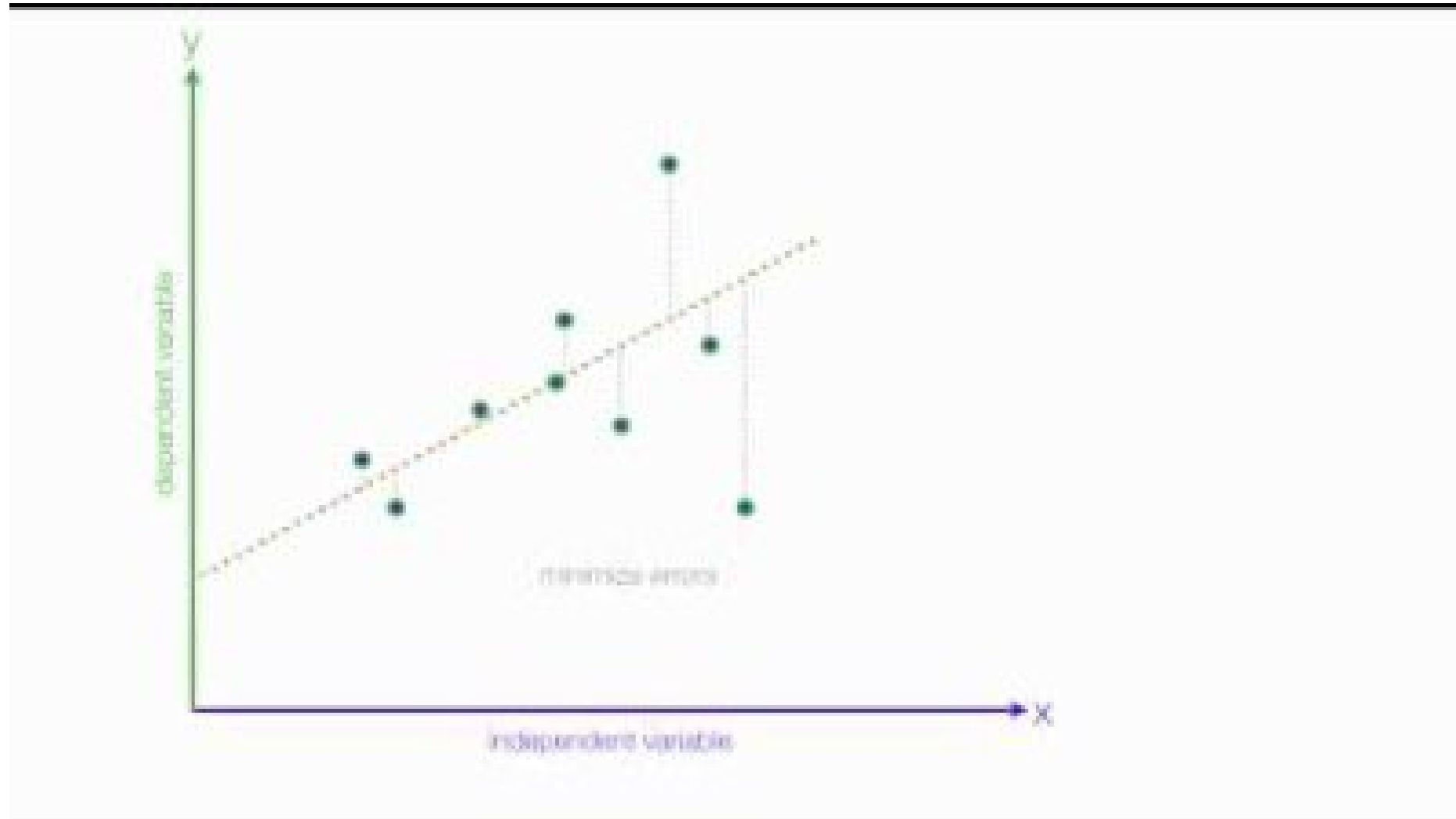
Correlation vs Regression.

- Compare correlation and regression.
- Is there any relationship between the correlation coefficient and the slope of the regression line in a simple linear regression?

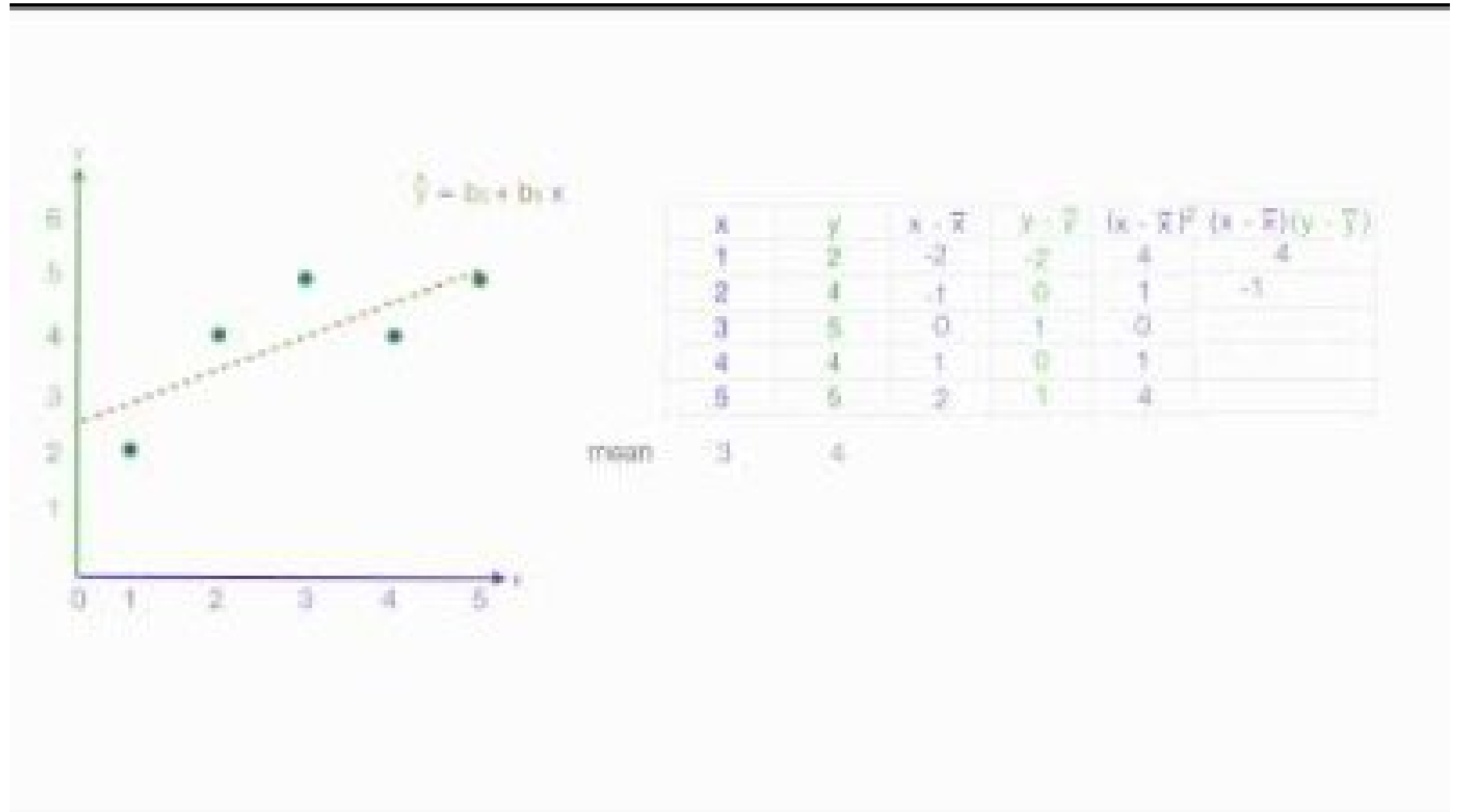
Ask ChatGPT 

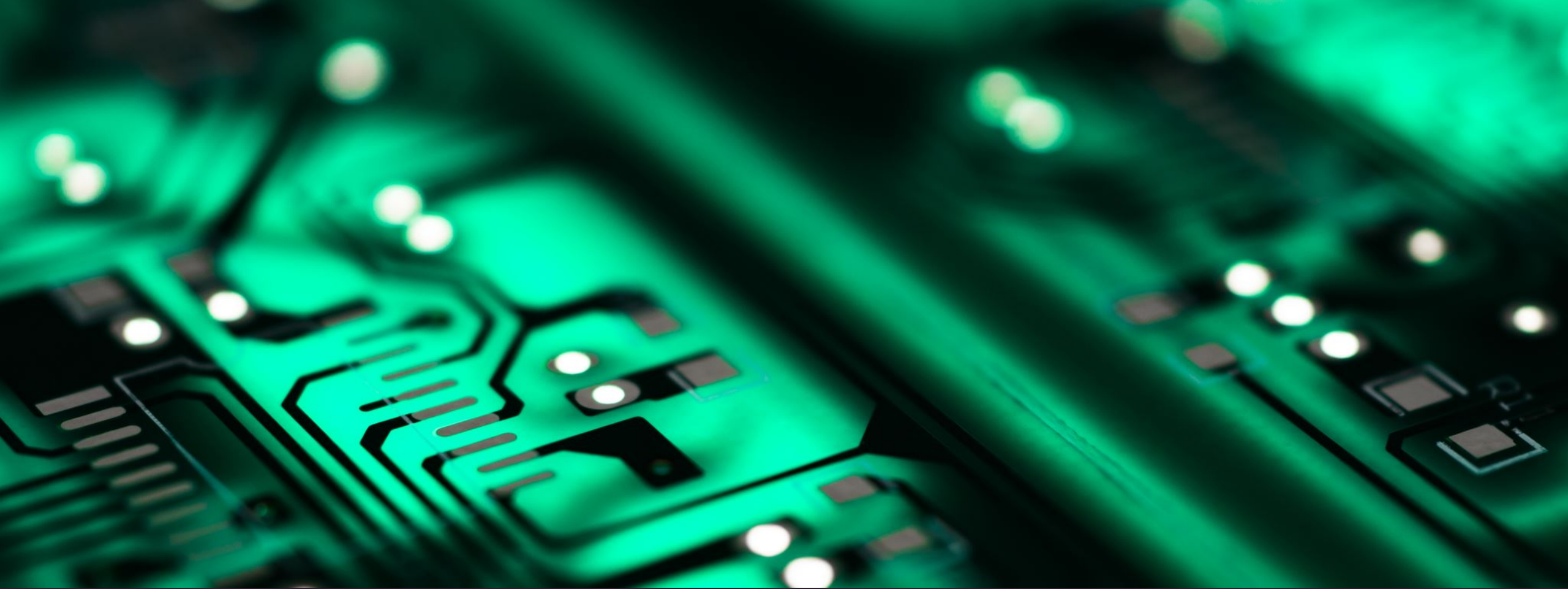


Movie Time 🌴



Movie Time 🌴

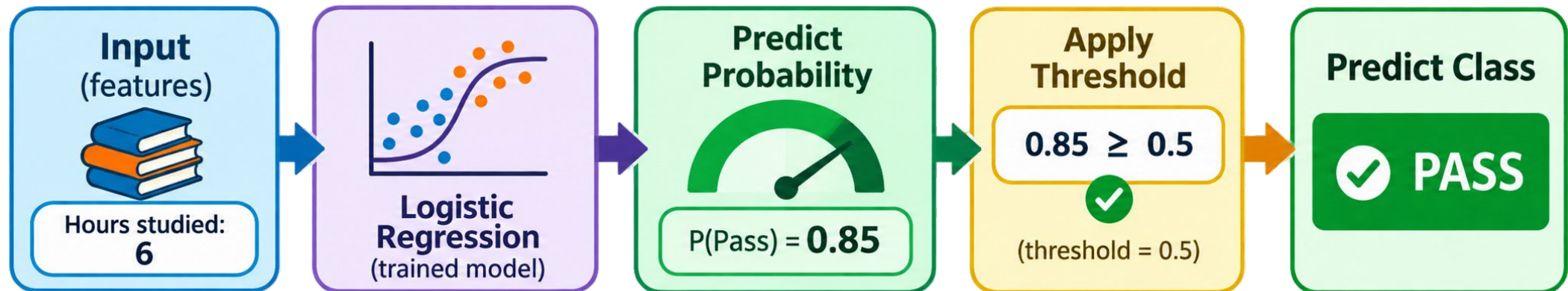




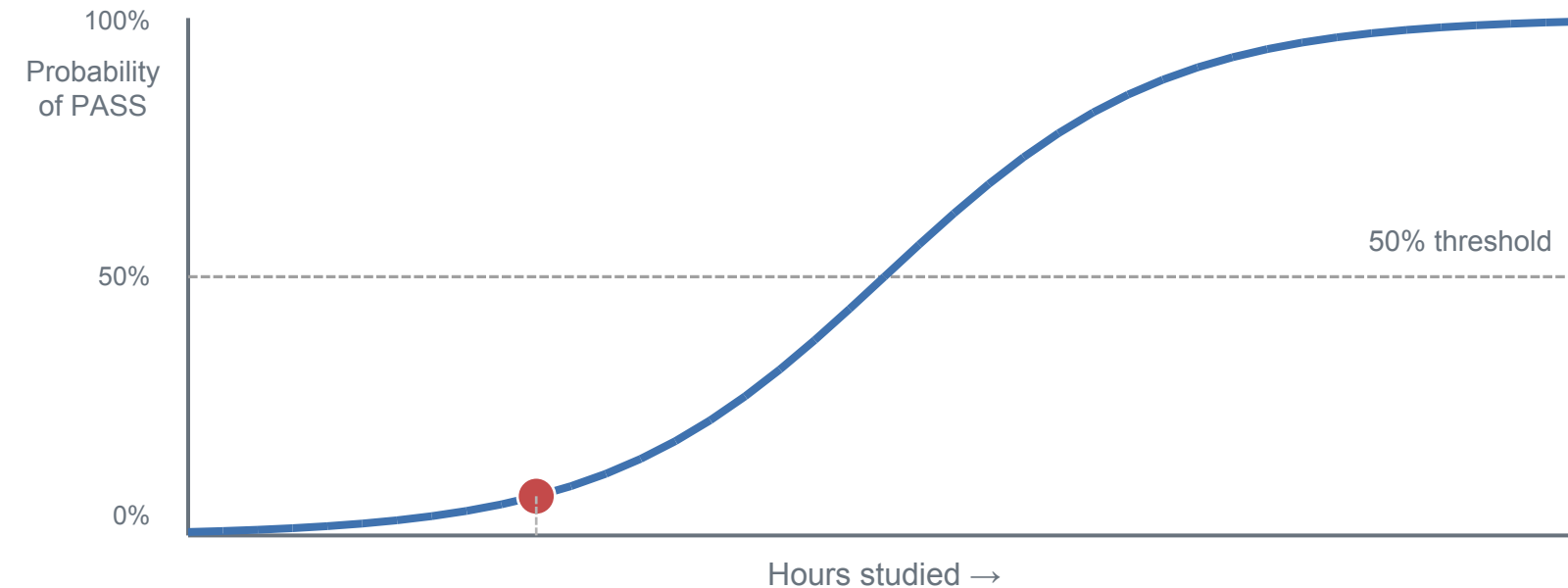
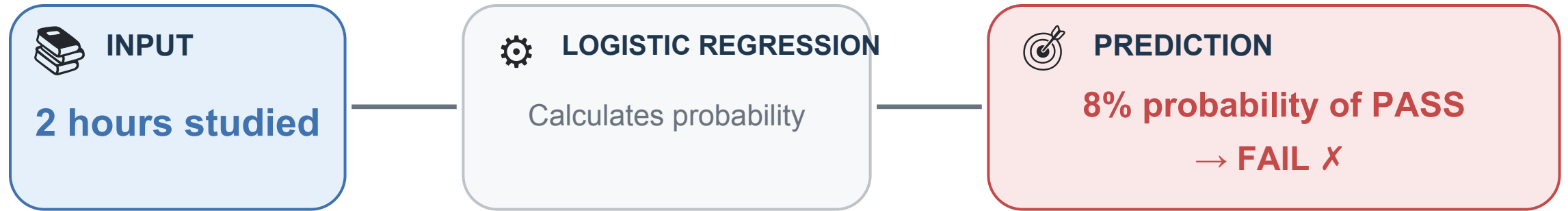
Logistic Regression

What is Logistic Regression?

- **Logistic regression** is a **supervised machine learning algorithm** used mainly for **classification**.
- Despite having the word “**regression**” in its name, it is usually used to predict **categories**, especially in two-class (binary, Y/N, T/F) problems.
- The model estimates the probability of the outcome occurring based on the values of the predictor variables.



Logistic Regression: From Input to Class



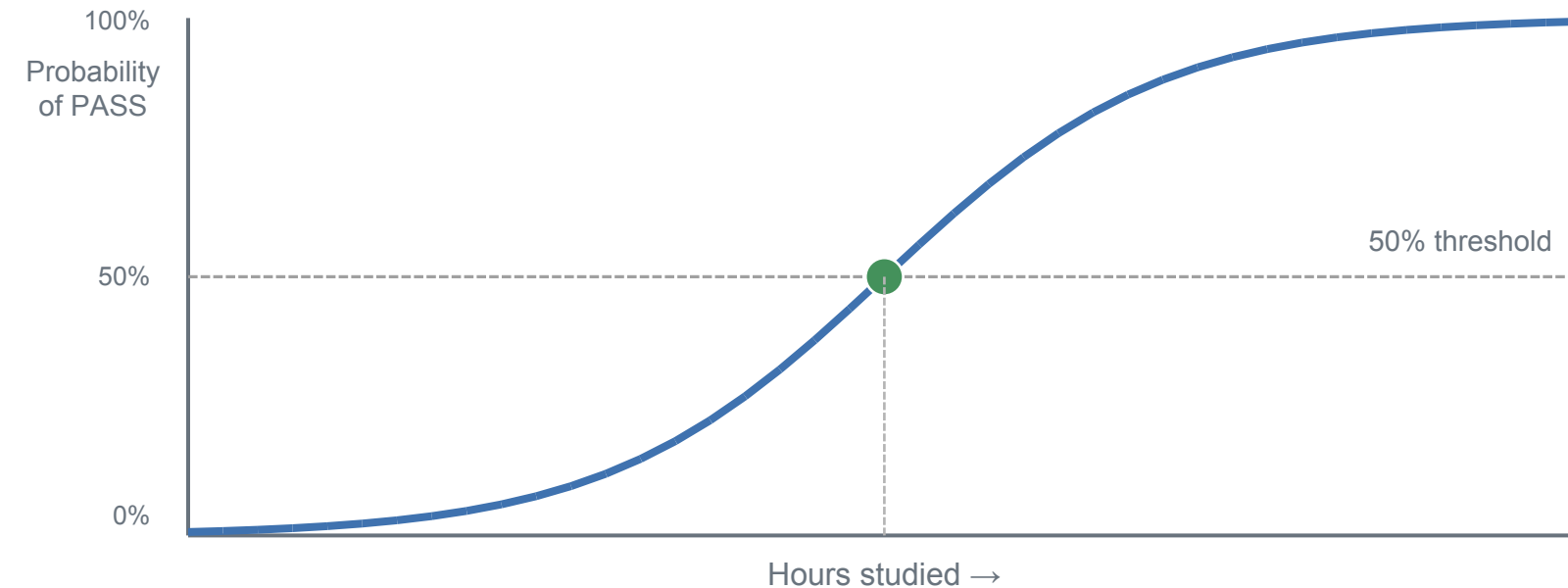
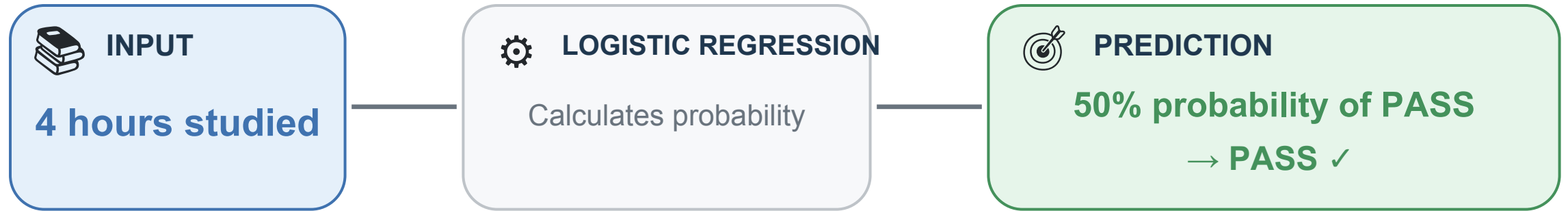
Decision

Probability = 8%

$8\% < 50\%$

Predict: FAIL

Logistic Regression: From Input to Class



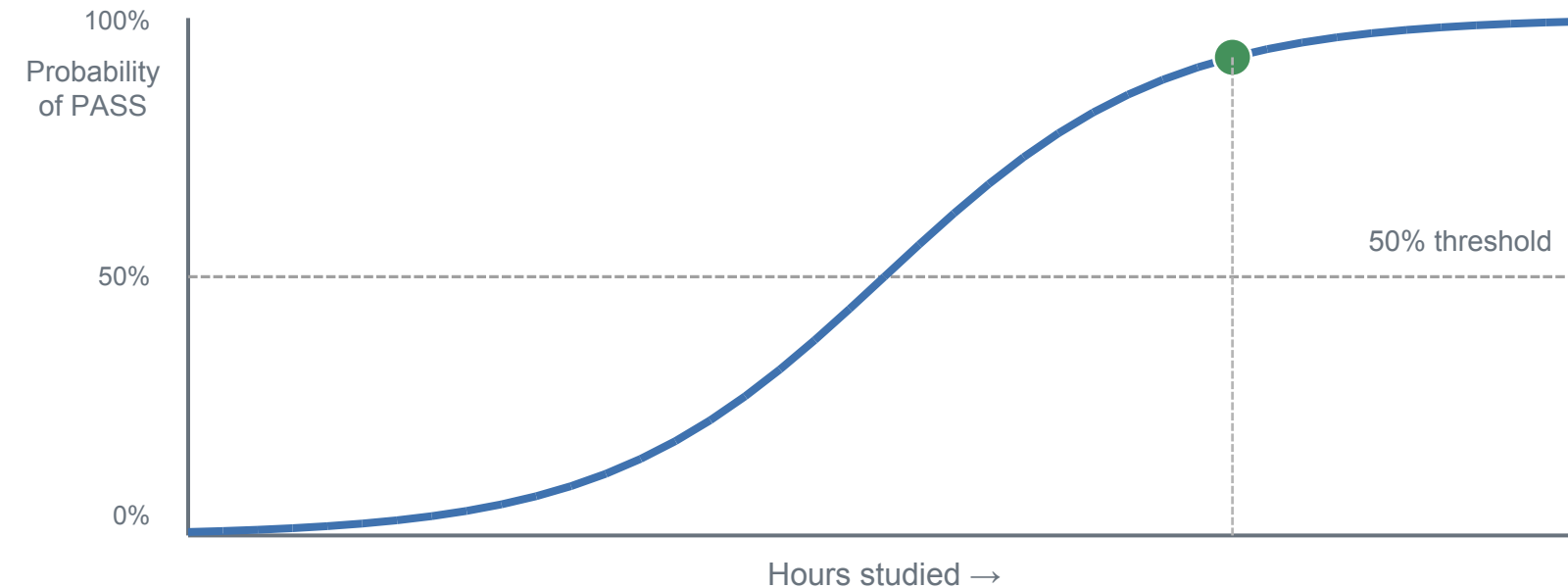
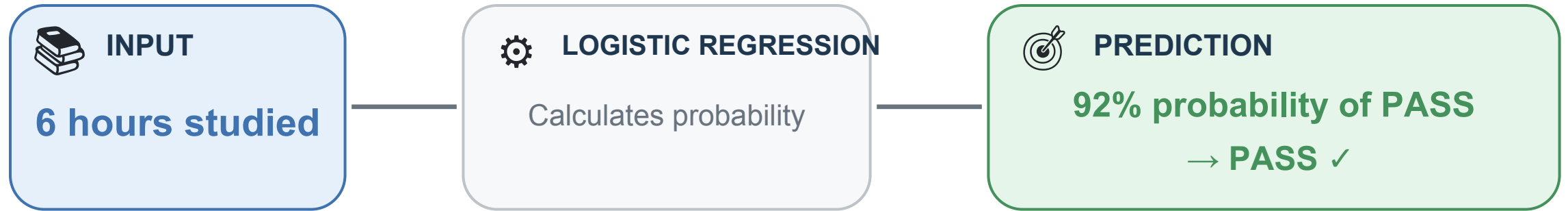
Decision

Probability = 50%

50% ≥ 50%

Predict: PASS

Logistic Regression: From Input to Class



Decision

Probability = 92%

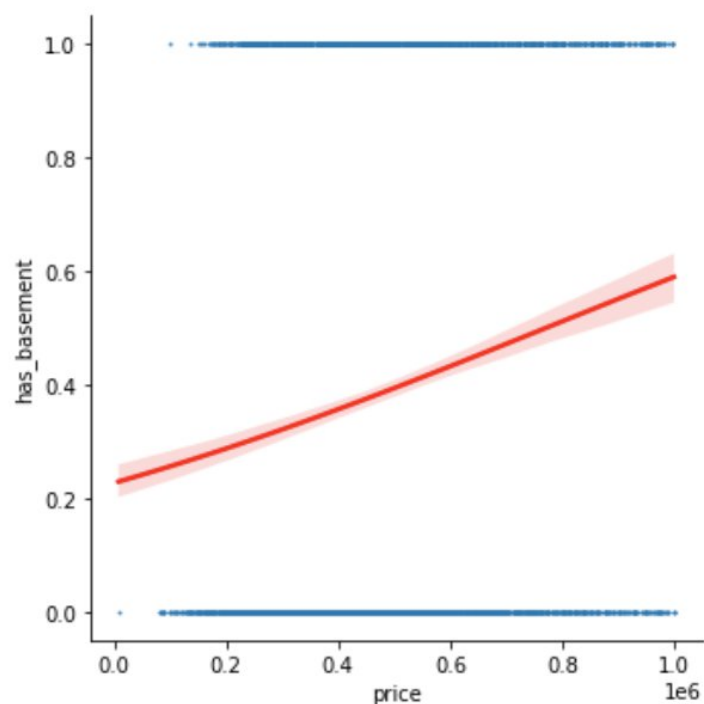
$92\% \geq 50\%$

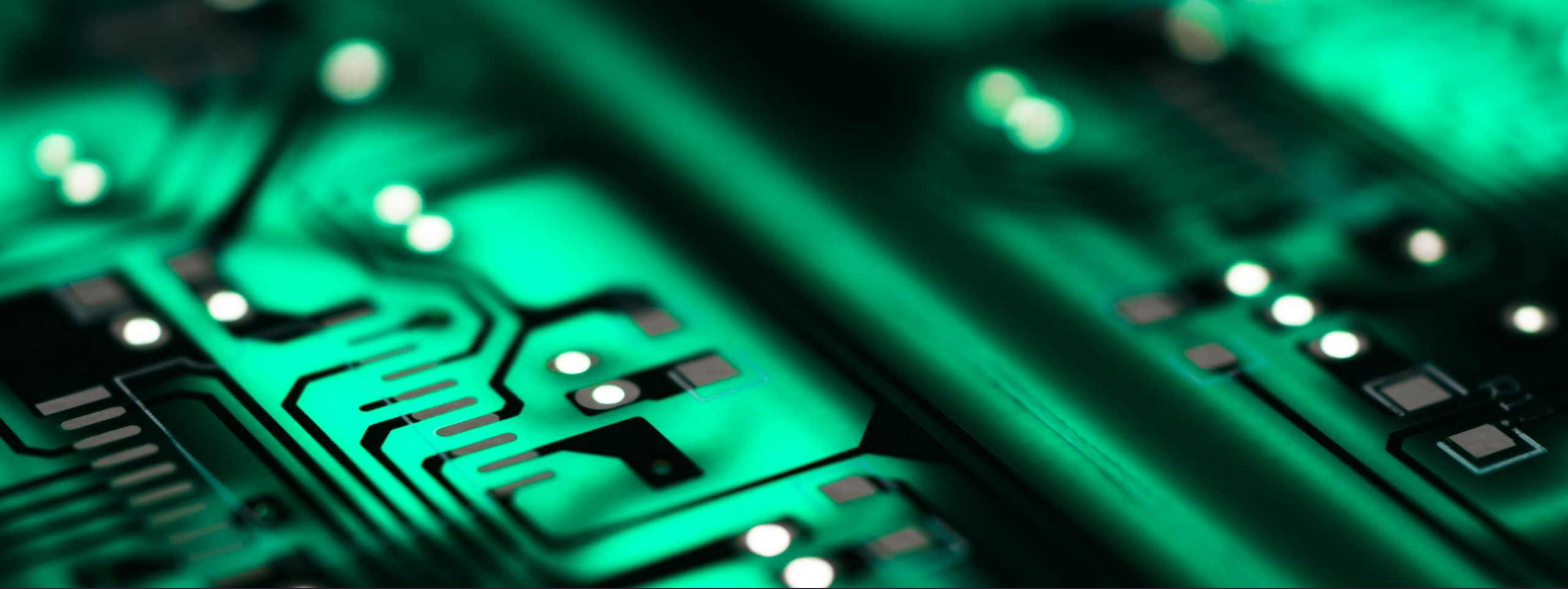
Predict: PASS

Plotting a Logistic Regression

```
# Create a logistic regression plot with a red line using Seaborn
sns.lmplot(data=housing, # The dataset to use
           x='price',     # The column to use for the x-axis
           y='has_basement', # The column to use for the y-axis
           ci=95,         # Set the size of the confidence interval to 95%
           scatter_kws={'s':1}, # Additional scatter plot settings (size of markers)
           line_kws={'color':'red'}, # Additional regression line settings (color)
           logistic=True); # Use logistic regression instead of linear regression
```

The logistic regression line models the probability of having a basement given the price of the house.
The x-axis shows the prices, and the y-axis shows whether the house has a basement or not (0 or 1).
The red line represents the logistic regression model,
which estimates the probability of having a basement based on the house price.
As the price increases, the probability of having a basement also increases,
up to a certain point where the probability levels off.
The ci parameter sets the size of the confidence interval for the logistic regression line.
A confidence interval is a range within which the regression line is likely to fall,
with a specified level of confidence.
In this case, the ci parameter is set to 95%, indicating that we are 95% confident
that the regression line falls within the specified range.





Polynomial Regression

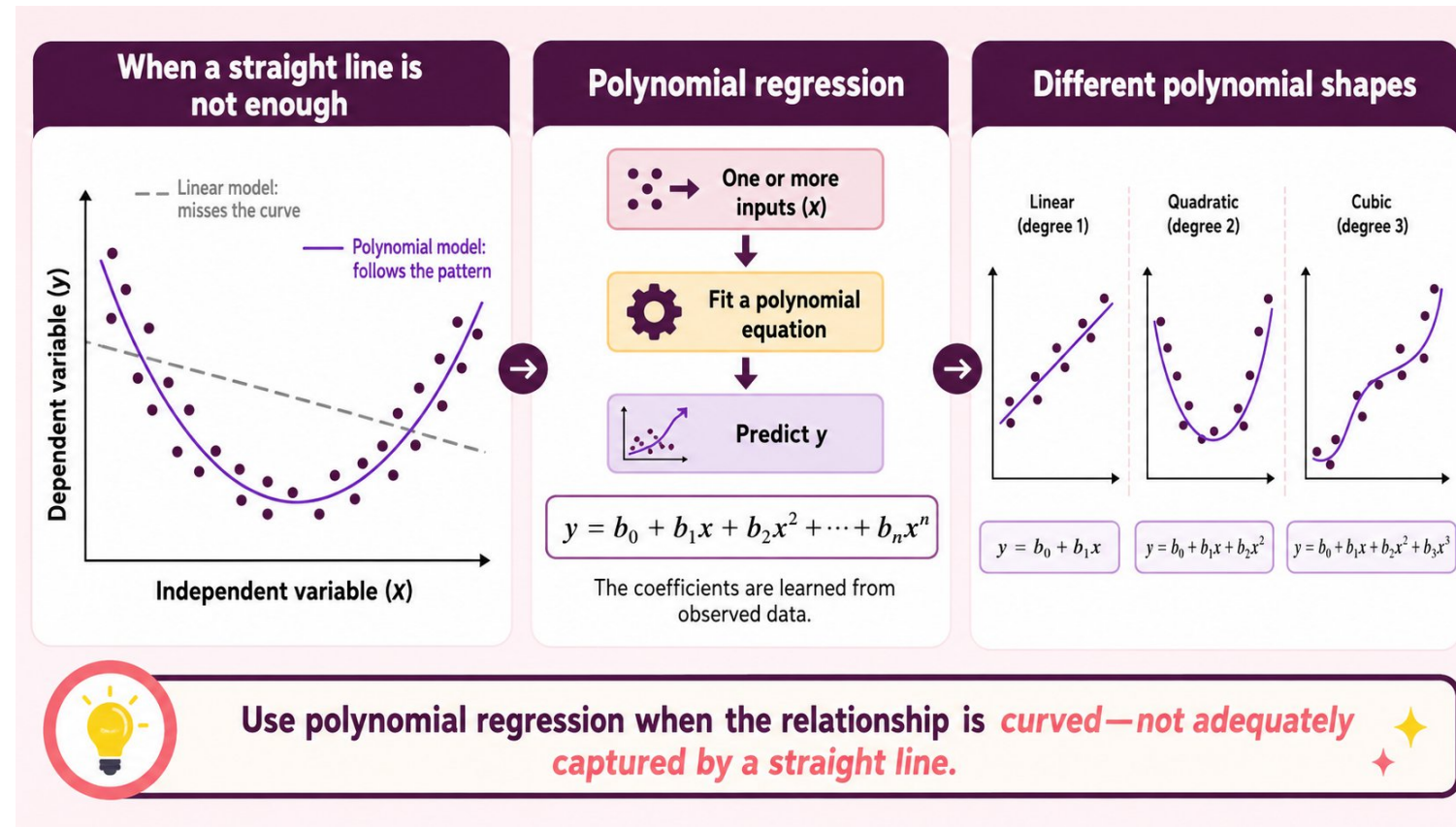
What is Polynomial Regression?

Used to model the relationship between a dependent variable (y) and **one or more independent variables (x)** by fitting a polynomial equation to the observed data.

One input: $y = b_0 + b_1x + b_2x^2 + \dots$

Two or more inputs:

$$y = b_0 + b_1x_1 + b_2x_1^2 + b_3x_2 + \dots$$



Plotting a Polynomial Regression

```
# Create a polynomial regression plot using Seaborn
sns.lmplot(x='Year',          # The column to use for the x-axis
           y='DeathRate',    # The column to use for the y-axis
           hue='AgeGroup',   # Group the data by AgeGroup and color-code the groups
           data=mortality_data, # The dataset to use
           ci=None,          # Do not show confidence intervals for the regression line
           markers='d',      # Set the marker style for the scatter plot
           x_bins=20,        # Divide the x-axis into 20 evenly-spaced bins
           order=3)          # Fit a cubic (degree 3) polynomial regression line
```

The regression line is computed using a polynomial function of the form $y = b_0 + b_1x + b_2x^2 + \dots + b_nx^n$, where n is the degree of the polynomial. A polynomial regression line can fit the data better than a linear regression line if there is a nonlinear relationship between the x and y variables.

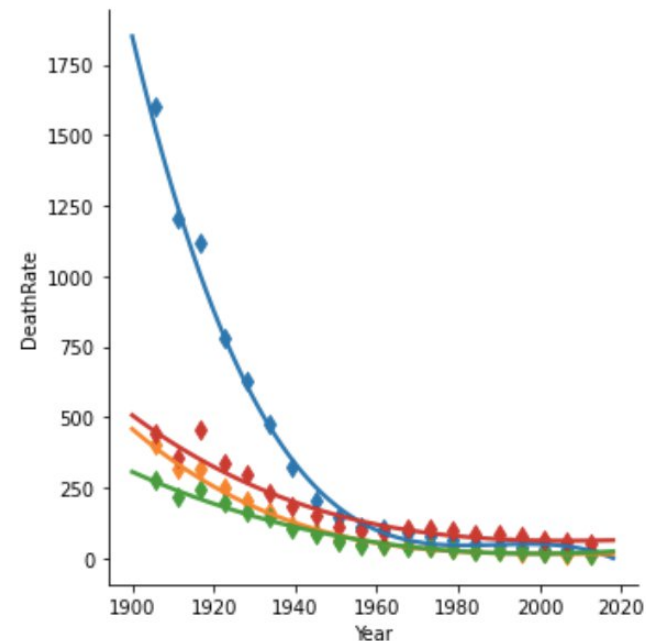
The hue parameter is used to group the data by AgeGroup and color-code the groups. This allows us to see if the relationship between Year and DeathRate varies by age group.

The markers parameter is used to set the marker style for the scatter plot. In this case, we are using diamond markers (denoted by 'd').

The x_bins parameter is used to divide the x-axis into 20 evenly-spaced bins. This can make it easier to see any patterns or trends in the data.

The order parameter is used to specify the degree of the polynomial regression line. In this case, we are fitting a cubic (degree 3) polynomial regression line. Higher-degree polynomial regression lines can fit the data even better, but they can also be more prone to overfitting.

<seaborn.axisgrid.FacetGrid at 0x14e8325f0>

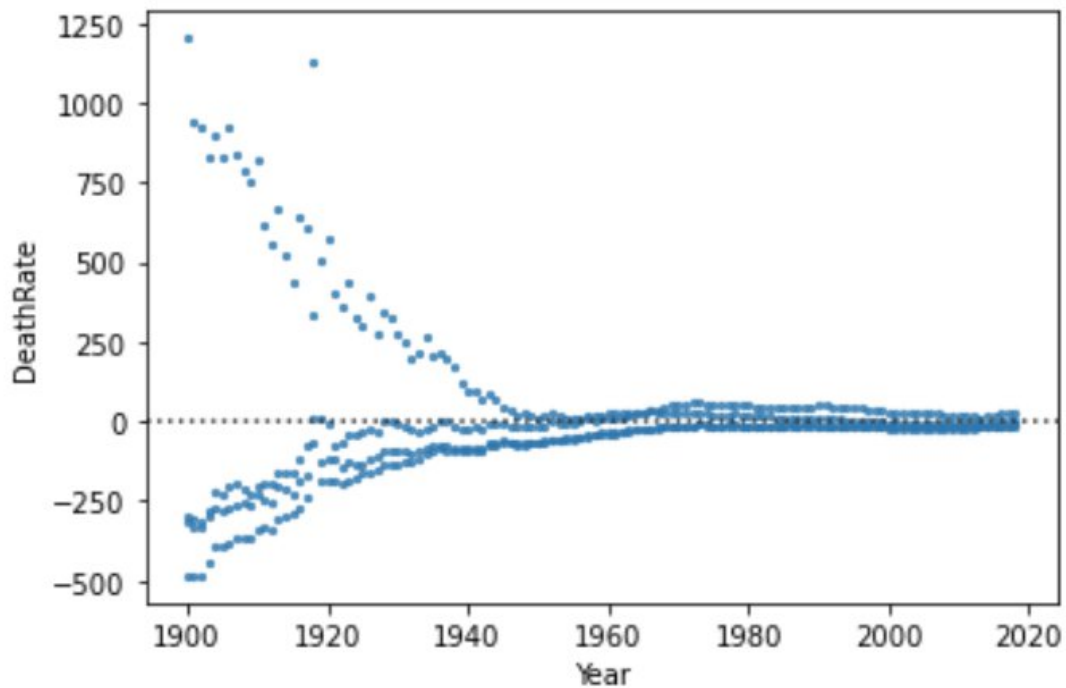


```
# Load mortality data from a pickle file and display the first few rows
mortality_data = pd.read_pickle('mortality_data.pkl')
mortality_data.head()
```

	Year	AgeGroup	DeathRate
0	1900	01-04 Years	1983.8
1	1901	01-04 Years	1695.0
2	1902	01-04 Years	1655.7
3	1903	01-04 Years	1542.1
4	1904	01-04 Years	1591.5

Plotting The Residuals for a Polynomial Regression

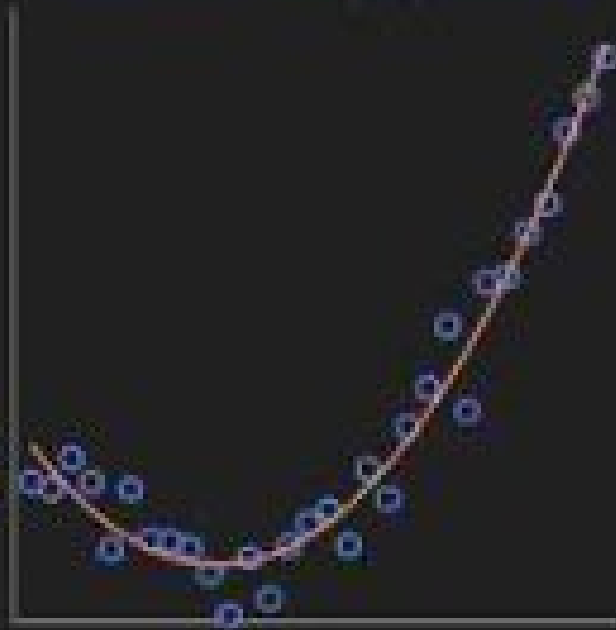
```
# Create a residual plot using Seaborn  
sns.residplot(data=mortality_data, x='Year', y='DeathRate', order=3, scatter_kws={'s':5});
```



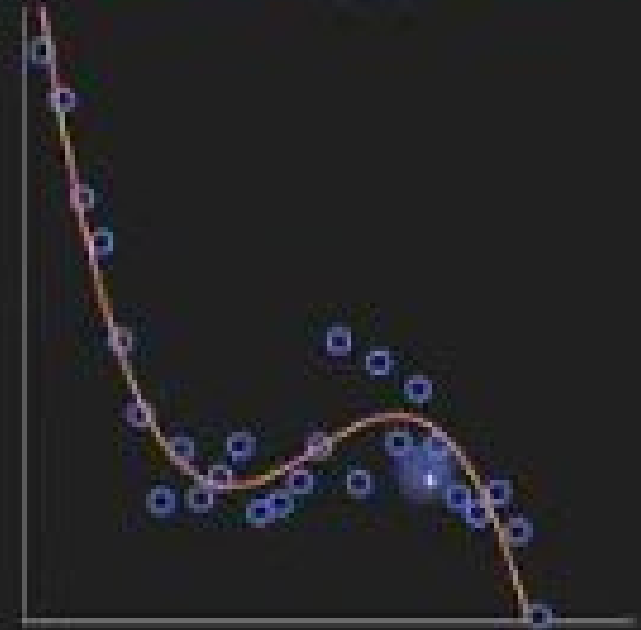
Movie Time 🌴

When would you use a polynomial regression?

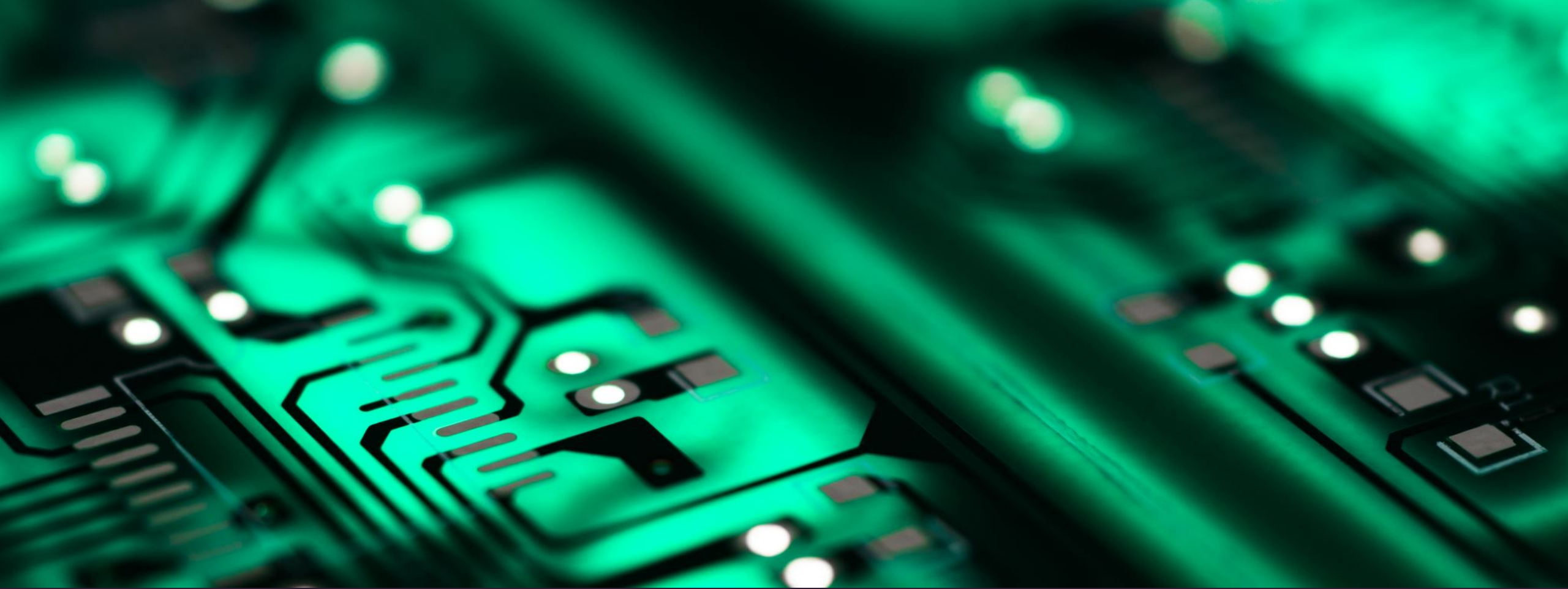
Second-order polynomial



Third-order polynomial



UNIVERSITY OF CALIFORNIA, BERKELEY



Lowess Regression

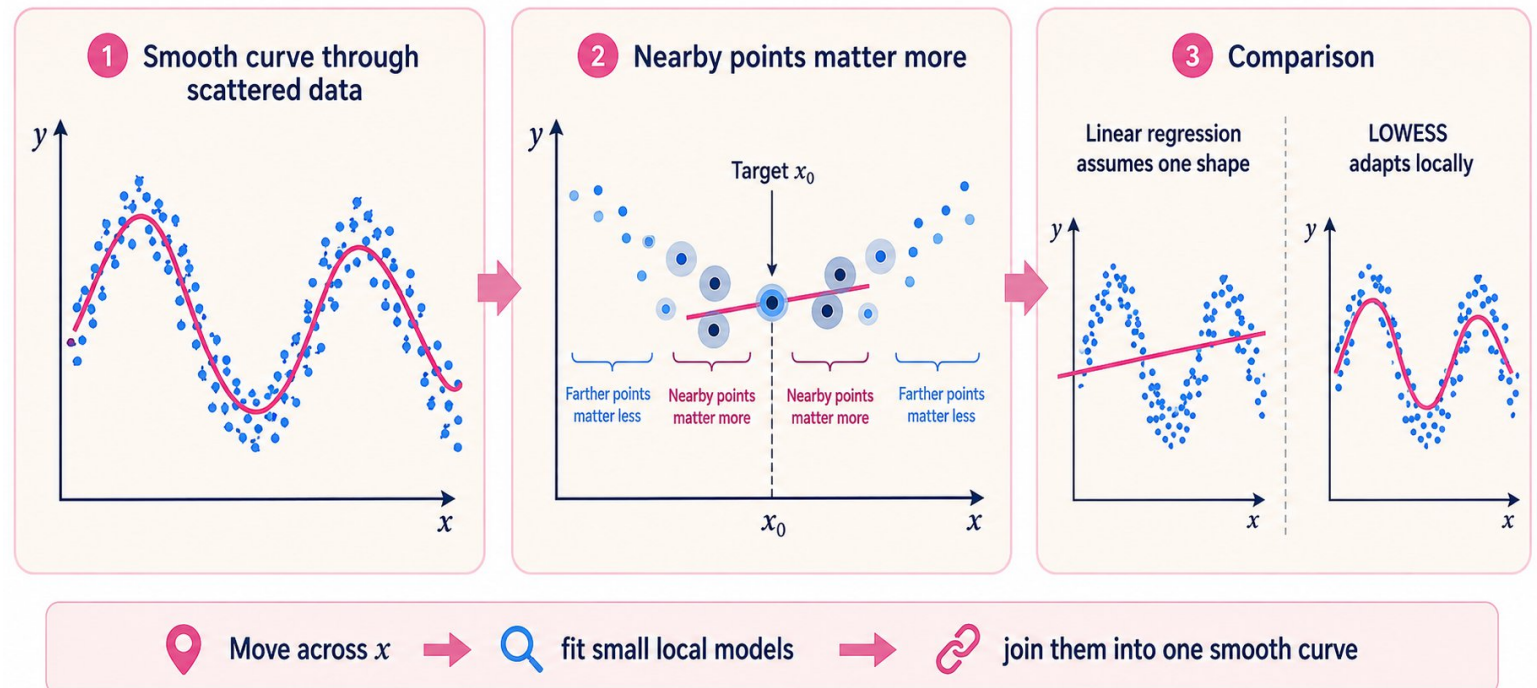
What is Lowess Regression?

- LOWESS (Locally Weighted Scatterplot Smoothing) regression is a **nonparametric regression** method used **to fit a smooth curve through a scatter plot of data points**. Unlike linear or polynomial regression, LOWESS does not assume one fixed equation for the entire dataset. It adapts to different local patterns, making it useful for exploring complex, nonlinear relationships.
- The weights used in the averaging process are based on the distance between the data point of interest and the other data points in the neighbourhood. LOWESS regression is particularly **useful when the relationship between the variables is complex or nonlinear**.
- **Important:** LOWESS is mainly used to reveal trends and patterns. It is less suitable for predicting far beyond the observed data range.

How does LOWESS work?

LOWESS looks nearby, fits locally and combines the local estimates into one smooth curve.

- Selecting one x -value at a time.
- Looking at the data points near that value.
- Giving **more weight to closer points** and less weight to distant points.
- Fitting a small regression model to that neighbourhood.
- Repeating the process across the dataset.
- Joining the local estimates to form one smooth curve.



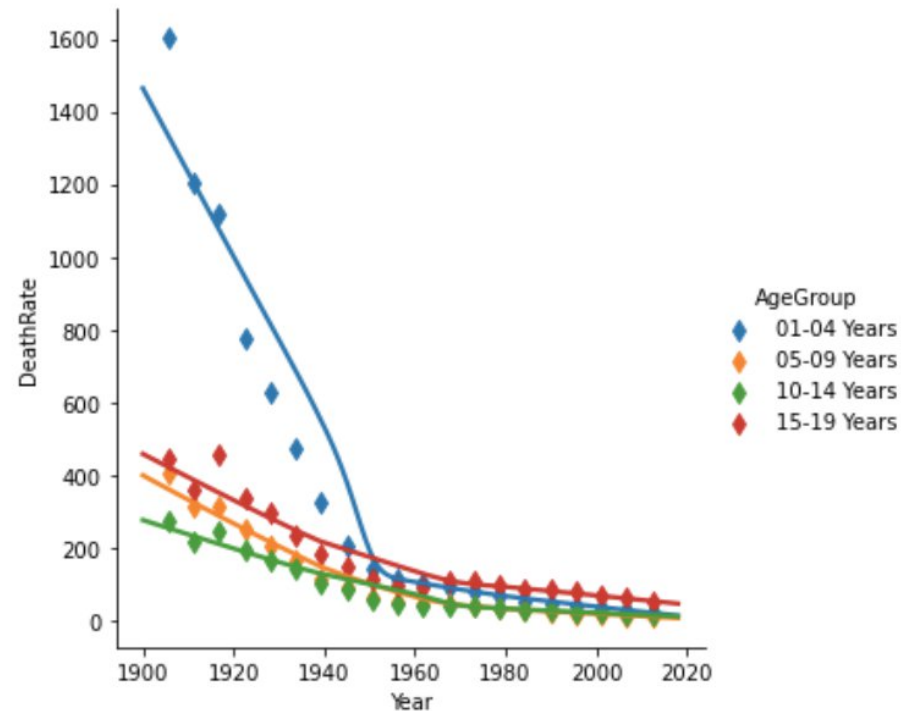
Plotting Lowess Regression

```
# Create a scatter plot with a regression line for each age group using Seaborn
sns.lmplot(data=mortality_data,
           x='Year',
           y='DeathRate',
           hue='AgeGroup',
           ci=None,
           markers='d',
           x_bins=20,
           lowess=True)

# The dataset to use
# The column to use for the x-axis
# The column to use for the y-axis
# Group the data by AgeGroup and show different colors
# The size of the confidence interval for the regression line
# The type of marker to use
# Number of bins to discretize the x-axis into
# Whether to use a locally weighted regression line (lowess)

# In summary, this code creates a scatter plot of Year vs DeathRate for mortality data,
# where each data point is colored based on the AgeGroup it belongs to.
# The plot includes a separate regression line for each age group.
# The ci parameter is set to None, indicating that no confidence interval is shown for the regression line.
# The markers are set to diamonds (d), and the x-axis is discretized into 20 bins. Finally, lowess is set to True,
# indicating that a locally weighted regression line should be used.
```

<seaborn.axisgrid.FacetGrid at 0x14cf616f0>



Linear regression	Polynomial regression	LOWESS
Fits one straight line	Fits one polynomial equation	Fits many small local models
Assumes a linear form	Requires choosing a degree	Does not require a fixed global form
Less flexible	More flexible	Highly flexible and data-driven
Supports an interpretable equation	Produces a polynomial equation	Does not produce one simple global equation

References

- Data science from scratch: first principles with Python, Joel Grus, O'Reilly Media, 2019
- Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython, Wes McKinney, O'Reilly Media, 3rd edition, 2022.
- Python Data Science Handbook: Essential Tools for Working with Data, Jake Vanderplas, O'Reilly Media, 2022
- Murach's Python for Data Analysis, Scott McCoy, Mike Murach & Associates, Incorporated, 2021.
- Intro to Python for Computer Science and Data Science: Learning to Program with AI, Big Data and The Cloud, Paul Deitel, Pearson Education Limited, 2021.
- ChatGPT



End of lecture ...